第1章: 環境構築 — 自分のPCで開発できるようにする（完全版）

> この章では、引き継いだ「業務工数（こうすう）システム」を、**あなたのWindows PCの中だけで動かせる状態**（＝開発環境）にするまでを、**1コマンドずつ・1行ずつ** 手取り足取り進めます。
>
> プログラミングがまったく初めてでも大丈夫です。「何を入れて」「何を打って」「どう表示されれば成功で」「失敗したらどう直すか」を、**実際の本アプリの設定ファイル（`requirements.txt` / `package.json` / `settings.py` / `.env.production`）の中身を本物のまま読み解きながら** 進めます。
>
> この章を最後までやり切ると、あなたのPCで `http://localhost:3000`（画面）と `http://localhost:8000`（裏方のサーバー）が動き、ログイン画面が表示されます。**これがゴールです。**
>
> 長い章ですが、**辞書のように使えること** を目指しています。途中で詰まったら、該当ステップとトラブルシューティング表に戻ってください。

この章で学ぶこと

- **開発環境とは何か** — 「サーバー」「ローカル」「フロントエンド／バックエンド」という言葉の意味
- **道具をそろえる** — VSCode（エディタ）／Git／Node.js v22／Python 3.12／PostgreSQL を1つずつインストールし、`--version` で確認
- **ソースコードを手元に持ってくる** — `git clone` の意味と実行
- **Python側（バックエンド）の準備** — `venv`（仮想環境）の作成・有効化、`pip install -r requirements.txt`、`requirements.txt` の全パッケージが何者か
- **`.env` ファイルを作る** — `SECRET_KEY` の生成、`DEBUG`、データベース接続情報。`settings.py` がこれをどう読むか1行ずつ
- **データベースの準備** — `migrate`（テーブル作成）、`createsuperuser`（管理者作成）
- **サーバーを起動する** — `runserver`、`qcluster`（非同期処理）
- **React側（フロントエンド）の準備** — `npm install`、`frontend/.env` と `REACT_APP_API_BASE_URL`、`npm start`
- **本番ビルド** — `npm run build` が何を作るか
- **豊富なトラブルシューティング** — つまずきポイントを症状別に網羅

目次

0. 前提知識：開発環境とは何か
1. 全体像：これから何を入れて、何を動かすのか
2. 道具をそろえる①：VSCode（エディタ）

3. 道具をそろえる②：Git
4. 道具をそろえる③：Python 3.12
5. 道具をそろえる④：Node.js v22
6. 道具をそろえる⑤：PostgreSQL
7. ソースコードを手元に持ってくる（git clone）
8. バックエンド準備①：仮想環境（venv）を作る
9. バックエンド準備②：requirements.txt 全解説とインストール
10. バックエンド準備③：.env を作る（SECRET_KEY生成含む）
11. settings.py を1行ずつ読む（.envとの対応）
12. バックエンド準備④：データベースを作って migrate
13. バックエンド準備⑤：管理者を作る（createsuperuser）
14. バックエンドを起動する（runserver）
15. 非同期ワーカーを起動する（qcluster）
16. フロントエンド準備①：package.json 全解説と npm install
17. フロントエンド準備②：frontend/.env と REACT_APP_API_BASE_URL
18. フロントエンドを起動する（npm start）
19. 本番ビルド（npm run build）
20. 毎日の起動手順まとめ（チートシート）
21. トラブルシューティング
22. 演習問題
23. この章のまとめ

---

0. 前提知識：開発環境とは何か

コマンドを打ち始める前に、これから自分が何をしようとしているのかを言葉で押さえておきましょう。ここが分かると、後で出てくる用語が「ただの呪文」ではなくなります。

0.1「開発環境」とは

> **開発環境（かいはつかんきょう）とは？** あなたのPCの中に、アプリを「作ったり・直したり・試したり」できる状態を整えること。料理でいえば「キッチンに、まな板・包丁・コンロ・冷蔵庫を全部そろえた状態」です。この章の作業は、まさにそのキッチン作りです。

本アプリを動かすには、大きく分けて **2つのプログラム** を自分のPCで同時に走らせます。

呼び名	担当	使う技術	起動URL（あなたのPC内）
フロントエンド	画面（見た目・操作）	React / TypeScript / Node.js	`http://localhost:3000`
バックエンド	裏方（データ・計算・保存）	Python / Django / PostgreSQL	`http://localhost:8000`

フロントエンド（front-end）とは？ ユーザーが直接見て触る「お店の表（フロント）」の部分。ボタン・入力欄・表など、画面のすべて。本アプリでは React で作られています。

バックエンド（back-end）とは？ ユーザーには見えない「お店の裏（バック）」の部分。データの保存・取り出し・計算をします。本アプリでは Python の Django で作られています。

たとえ話：レストランでいうと、**フロントエンド＝ホール（お客さんが座る席・メニュー）**、**バックエンド＝厨房（料理を作る場所）**、**データベース＝食材倉庫**。お客さん（あなた）が画面で「工数を保存」と注文すると、ホール（React）が厨房（Django）に伝え、厨房が倉庫（PostgreSQL）に食材（データ）を出し入れします。

0.2 「ローカル」と「サーバー」「localhost」

ローカル（local）とは？「あなたのPCの中」という意味。`localhost`（ローカルホスト）は「この自分のPC自身」を指す特別な住所（アドレス）です。`http://localhost:3000` は「自分のPCの中の3000番の窓口」という意味になります。

サーバー（server）とは？「要求（リクエスト）を受けて、応答（レスポンス）を返すプログラム」のこと。本アプリの Django も React も、起動すると「サーバー」として動き、ブラウザからの要求に画面やデータを返します。**サーバー＝特別な高性能マシン、とは限りません。**あなたのPC上で動く1つのプログラムも立派なサーバーです。

ポート番号（port number）とは？ 1台のPCの中で、複数のプログラムが同時に通信できるように割り振られた「窓口番号」。`3000` 番はReact、`8000` 番はDjango、`5432` 番はPostgreSQL……というように、プログラムごとに別々の番号を使います。郵便でいう「○○ビルの3階」「8階」のような部屋番号です。

0.3 この章のゴール（先に完成形を見る）

最終的に、あなたは次の状態を作ります。

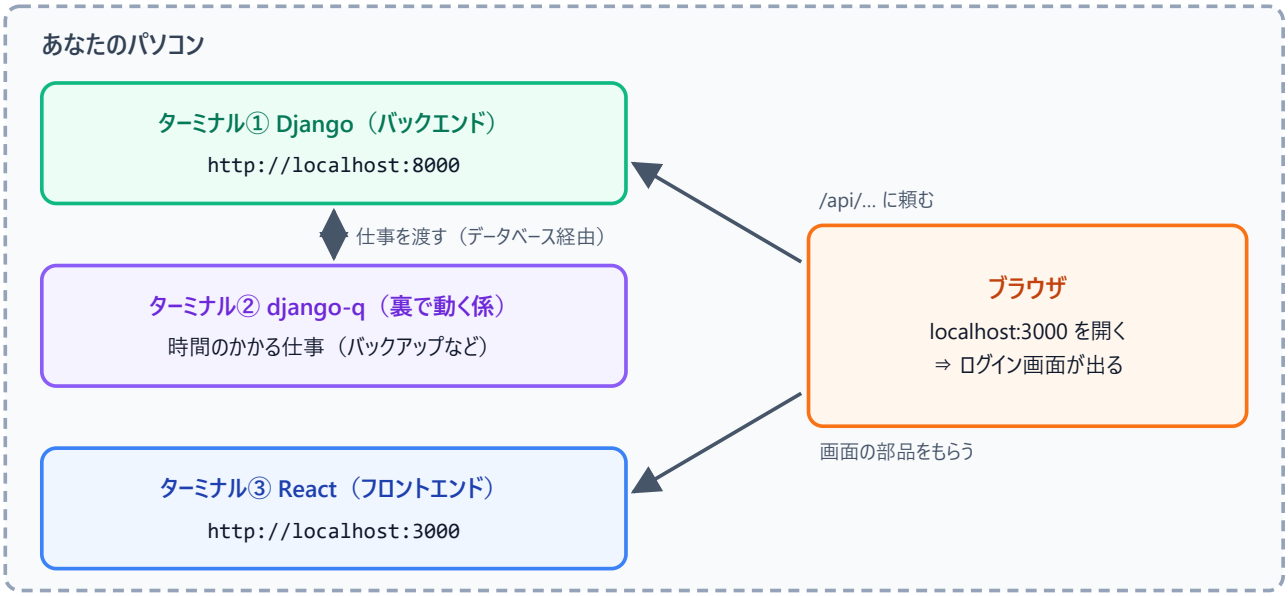


図1-1: 開発の時のパソコンの中。ターミナルを 3 つ開いて、それぞれでサーバーを動かし、ブラウザで localhost:3000 を開く

> **ターミナル（terminal）とは？** 文字でコマンドを打ってPCに命令する黒い画面（または白い画面）のこと。「コマンドプロンプト」「PowerShell（パワーシェル）」も同じ仲間です。本章では VSCode 内蔵のターミナルを使います。マウスでアイコンをクリックする代わりに、**文字で命令を出す道具** と思ってください。
>
> ⚠️ **この章を進める順番は飛ばさないでください。**「道具をそろえる→コードを取得→裏方を準備→画面を準備」という順番には意味があります。たとえばPythonを入れる前に `pip install` を打っても動きません。上から順にやれば必ずゴールにたどり着けるように作ってあります。

---

1. 全体像：これから何を入れて、何を動かすのか

まず、これから **インストールする5つの道具** を一覧で押さえましょう。それぞれ「なぜ必要か」も書きます。

#	道具	役割（家でたとえると）	なぜ必要か
1	**VSCode**	作業机・道具箱	コードを読み書きし、ターミナルも開くため
2	**Git**	写真の履歴管理＋取り寄せ機	ソースコードをダウンロードし、変更履歴を管理するため
3	**Python 3.12**	厨房の調理器具一式（裏方の言語）	Django（バックエンド）を動かすため
4	**Node.js v22**	別の調理器具一式（画面側の言語）	React（フロントエンド）を動かすため
5	**PostgreSQL**	食材倉庫（データベース本体）	工数・人員などのデータを保存するため

> **なぜ？ Python と Node.js の両方が要るの？** 本アプリは「画面（React=Node.js製）」と「裏方（Django=Python製）」という、**2つの異なる言語で書かれた2つのプログラムの合体** だからです。片方だけでは動きません。これは珍しいことではなく、現代の多くのWebアプリがこの「フロント＝JavaScript系、バック＝Python/Java/PHP等」という二刀流構成です。

1.1 バージョン（version）をそろえる重要性

> **バージョン（version）とは？** ソフトの「版」を表す番号。`Python 3.12` 、`Node.js v22` のように書きます。新しいほど機能が増えますが、**指定とずれると動かない** ことがあります。

本アプリで使うバージョンは次の通りです。**できるだけこれに合わせてください。**

- **Python … 3.12 系**（本番へ出す自動の仕組み＝GitHub Actions の設定ファイル `.github/workflows/master_hozen-kosu-react.yml` に `python-version: "3.12"` とあります）
- **Node.js … v22 系をおすすめ**（`frontend/` フォルダに `node-v22.17.1-linux-x64.tar.xz` という Node.js v22.17.1 の Linux 用の一式が置いてあります。なお、GitHub Actions の画面のビルドは `node-version: "20"` 、つまり v20 で行っています。v20 でも動きます）
- **PostgreSQL … 任意の新しめの版**（14以降を推奨）

> ⚠️ **バージョン違いは初心者最大の落とし穴です。** 特に Python は、3.12 と違う版（たとえば 3.8 のような古い版）だと、`pip install -r requirements.txt` で部品が入らないことがあります。Node.js は v20 か v22 を使い、それより古い v18 以前は使わないでください（v18 は 2025 年 4 月で手入れ（サポート）が終わっています）。迷ったら「v22」を選びます（§5で手順を詳述）。

---

2. 道具をそろえる①：VSCode（エディタ）

> **VSCode（ブイエス・コード）とは？** Visual Studio Code（ビジュアル・スタジオ・コード）の略。Microsoft 製の無料の **コードエディタ**（コードを読み書きする高機能なメモ帳）。世界で最も使われている開発用エディタで、ターミナルも内蔵しています。本章のコマンドはすべて、この中のターミナルで打ちます。

2.1 インストール手順

1. ブラウザで `https://code.visualstudio.com/` を開く。
2. 大きな青いボタン「**Download for Windows**」をクリック。
3. ダウンロードされた `VSCodeUserSetup-x64-x.x.x.exe` をダブルクリック。

4. インストーラの指示に従って進む。途中の「**追加タスクの選択**」画面では、次の2つに **チェックを入れる** と便利です。 - ☑「PATH への追加」（これで `code` コマンドが使えるようになる） - ☑「エクスプローラーのファイル／ディレクトリのコンテキストメニューに『Code で開く』を追加」

> **PATH（パス）とは？**「コマンドを打ったとき、PCがプログラムを探しに行く場所のリスト」。ここに登録されていないと、`python` や `git` と打っても「そんなコマンド知らない」と言われます。インストーラの「PATH に追加」にチェックを入れると、自動で登録してくれます。**迷ったらチェックを入れる、が正解です。**

2.2 日本語化（任意・おすすめ）

VSCode は最初は英語です。日本語にしたい場合：

1. VSCode を起動し、左端の四角が4つ並んだアイコン（**拡張機能**）をクリック。
2. 検索欄に `Japanese Language Pack` と入力。
3. 「Japanese Language Pack for Visual Studio Code」の **Install** を押す。
4. 右下に「再起動して言語を変更しますか？」と出たら **Restart**。

> **▼ こう表示されれば成功:** VSCode のメニューが「ファイル」「編集」「表示」…と日本語になります。

2.3 内蔵ターミナルの開き方（最重要）

本章で何度も使うので、覚えてください。

- メニューの「**ターミナル**」→「**新しいターミナル**」
- または ショートカット `Ctrl + Shift + @`（半角・全角キーの近くにある記号）

> **▼ こう表示されれば成功:** VSCode の下半分に黒っぽい領域が開き、`PS C:\...>` のような文字（プロンプト）が出ます。ここに文字を打って `Enter` でコマンドを実行します。
>
> **用語: プロンプト（prompt）** … ターミナルで「ここに入力してください」と待っている記号や行のこと。Windows の PowerShell では `PS C:\Users\あなた>` のように表示されます。本書では入力例から `PS C:\...>` の部分を省略し、**打つコマンドだけ** を書きます。

---

3. 道具をそろえる②：Git

> **Git（ギット）とは？** ソースコードの **変更履歴を記録・管理** する道具。「いつ・誰が・どこを・どう変えたか」を全部覚えてくれる、超高機能なセーブ機能です。さらに、GitHub などに置かれたコードを **手元にダウンロード（clone）** する役目も果たします。Git の詳しい使い方は次章「02_Git_GitHub.md」で扱います。この章では「インストールして、コードを1回ダウンロードできれば十分」です。

3.1 インストール手順

1. ブラウザで `https://git-scm.com/download/win` を開く。
2. 自動でダウンロードが始まる（始まらなければ「64-bit Git for Windows Setup」をクリック）。
3. `Git-x.xx.x-64-bit.exe` をダブルクリック。
4. インストーラは設定項目が多いですが、**基本はすべて既定（デフォルト）のまま「Next」を連打でOK** です。初心者は迷わず既定で進めてください。
5. 最後に「Install」→「Finish」。

> ⚠️ **VSCode を開いたままGitをインストールした場合は、VSCode を一度閉じて開き直してください。**そうしないと、VSCode のターミナルが新しく入った `git` コマンドを認識しないことがあります（PATH の読み込みは起動時に行われるため）。

3.2 インストール確認

VSCode のターミナルを開き、次を打って `Enter`。

> ▼ **このコマンドがやること:** インストールされた Git の版を表示します。「ちゃんと入ったか」の確認です。

コピー

```
# gitのバージョン（版番号）を表示する
git --version
```

> ▶ **ここで初めて出てくるコード**
>
> - **`git`**（Git）：ファイルの変更の記録を残す道具（Git）の本体のコマンド。`git --version` で版を確かめられます。`git status`・`git add`・`git commit` のように、後ろに「何をするか」を付けて使います。
> - **`git --version`**（Git）：`git --version` は、入っている Git の版を表示します。
> - **`#`**（PowerShell）：`#` から行の終わりまでは「コメント」（人が読むためのメモ）で、PowerShell は無視します。コマンドの横に、何をするかを日本語で書いておけます。コピーしてそのまま貼り付けても、問題ありません。

▼ こう表示されれば成功:

コピー

```
git version 2.45.2.windows.1
```

（数字は違ってかまいません。`git version ...` と出れば成功です。）

`--version` とは？ 多くのコマンドに付けられる共通オプションで、「そのプログラムの版を表示して終わる」という意味。**「ちゃんとインストールできたか」を確認する定番の合言葉** です。この章では Git・Python・Node など、入れるたびにこれで確認します。

▼ こう出たら失敗（未インストール／PATH未登録）:

コピー

```
git : 用語 'git' は、コマンドレット、関数、スクリプト ファイル、または操作可能なプログラムの名前として認識されません。
```

→ §3.1のインストールをやり直すか、VSCode を再起動してください（§21トラブルシューティング参照）。

3.3 初回だけ：名前とメールを登録

Git は「誰が変更したか」を記録するため、名前とメールアドレスを覚えさせます。**最初の1回だけ** でOKです。

▼ このコマンドがやること: これから記録する変更の「作者名」と「作者メール」を、このPC全体（`--global`）に設定します。

コピー

```
# 作者名を設定（自分の名前に置き換える）
git config --global user.name "Yamada Taro"
# 作者メールを設定（自分のメールに置き換える）
git config --global user.email "you@example.com"
```

▶ ここで初めて出てくるコード

- **`git config`**（Git）: Git の設定を見る・変えるコマンド。`git config --global user.name "名前"` で、記録に残す名前を決めます。`--list` で、今の設定を一覧で見られます。
- **`git config --global`**（Git）: `--global` は、自分（このユーザー）の、全部のリポジトリに効く設定を対象にする指定です。`git config --global user.name "名前"` のように、最初に 1 回だけ設定します。

- `git config` ... Git の設定を変えるコマンド。
- `--global` ...「このPC全体に共通の設定」という意味（特定のプロジェクトだけでなく全体に効く）。

- `user.name` / `user.email` … 設定する項目の名前。

> ▼ **確認:** 次で設定が反映されたか見られます。
>
> コピー
>
> ```
> git config --global user.name    # 設定した名前を表示
> ```
>
> あなたが入れた名前が表示されれば成功です。

---

4. 道具をそろえる③：Python 3.12

> **Python（パイソン）とは？** 読みやすさを重視したプログラミング言語。本アプリの **バックエンド（裏方）** は、このPython で書かれた **Django（ジャンゴ）** というフレームワークで作られています。
>
> **フレームワーク（framework）とは？**「アプリの土台・骨組みのセット」。一からすべて書かなくても、よくある機能（データベース接続・URL振り分け・管理画面など）が最初から用意された道具箱です。Django は Python 製の代表的なWebフレームワークです。

4.1 インストール手順

1. ブラウザで `https://www.python.org/downloads/` を開く。
2. **3.12 系** をダウンロードします。トップの大きなボタンが 3.13 など新しすぎる版になっている場合は、ページを下にスクロールし「Looking for a specific release?」一覧から **Python 3.12.x**（例: 3.12.7）の「Download」→ Windows の「Windows installer (64-bit)」を選びます。
3. ダウンロードした `python-3.12.x-amd64.exe` をダブルクリック。
4. **ここが最重要です。** インストーラ最初の画面の一番下にある「**Add python.exe to PATH**」（**PATHに追加**）**に必ずチェックを入れてください。**

> ⚠️ **「Add python.exe to PATH」のチェックを忘れると、ターミナルで `python` と打っても動きません。** 初心者が最も多くつまずく箇所です。チェックを入れ忘れてインストールしてしまったら、一度アンインストールして入れ直すのが確実です。

5. チェックを入れたら「**Install Now**」をクリック。
6. 完了画面で「Setup was successful」と出たら「Close」。

4.2 インストール確認

VSCode を一度閉じて開き直し（PATH反映のため）、ターミナルで次を打ちます。

▼ **このコマンドがやること:** インストールされた Python の版を表示します。

コピー

```
# Pythonのバージョンを表示
python --version
```

▶ **ここで初めて出てくるコード**

- **`python`**（Python の道具）: Python を動かすコマンド。`python -c "…"` は、`"…"` の中の Python のコードを 1 回だけ動かします。
- **`python --version`**（Python の道具）: 入っている Python の版（バージョン）を表示します。`Python 3.12.x` のように出ます。

▼ **こう表示されれば成功:**

コピー

```
Python 3.12.7
```

（`3.12.` で始まっていればOK。末尾の数字は違ってかまいません。）

⚠️ **Windows特有の罠：`python` と打つと Microsoft Store が開いてしまう場合** Windows には「`python` と打つと Store のインストール画面を開く」というダミー設定が初期から入っていることがあります。本物の Python が `Python 3.12.x` と表示されず、Store が開く場合は、§21のトラブルシューティングを参照してください（「アプリ実行エイリアス」をオフにします）。

4.3 pip の確認

pip（ピップ）とは？ Python の **パッケージ（部品ライブラリ）をインストールする道具**。Python本体に付属しています。§9で `pip install` を使って Django などを一括インストールします。

コピー

```
# pipのバージョンを表示
pip --version
```

▶ ここで初めて出てくるコード

- **`pip`**（Python の道具）: Python の部品（ライブラリ）を入れたり消したりする道具。`pip install 部品の名前` で入れます。
- **`pip --version`**（Python の道具）: pip の版と、どの Python の pip かを表示します。仮想環境に入っていると、表示される場所に `venv` が入ります。

▼ こう表示されれば成功:

コピー

```
pip 24.0 from C:\...\Python312\lib\site-packages\pip (python 3.12)
```

末尾に `(python 3.12)` と出ていれば、3.12 用の pip が使えている証拠です。

---

5. 道具をそろえる④：Node.js v22

Node.js（ノード・ジェイエス）とは？ 本来ブラウザの中でしか動かなかった JavaScript を、**PC上（ブラウザの外）でも動かせるようにする実行環境**。本アプリの **フロントエンド（React）の開発** には、この Node.js が必須です。React のコードを、ブラウザが分かる形に変換（ビルド）したり、開発用サーバーを動かしたりするのに使います。

npm（エヌ・ピー・エム）とは？ Node Package Manager の略。Node.js に付属する、**JavaScript のパッケージ（部品）をインストールする道具**。Python の pip にあたります。§16で `npm install` を使い、React などを一括インストールします。

5.1 インストール手順（v22 を選ぶ）

1. ブラウザで `https://nodejs.org/` を開く。
2. 2つのボタンが出ます。「**22.x.x LTS**」と書かれた方を選んでください。

LTS（エルティーエス）とは？ Long Term Support（長期サポート）の略。「長く安定して使える、おすすめの安定版」という意味。開発では原則 LTS を選びます。本アプリは v22 系を想定しているので、「**22.x.x LTS**」を選びます。

3. もしトップに 22 系が見当たらない場合は、ページの「**Other Downloads**」や「**Previous Releases**」から **v22 系**（例: v22.17.1）の「Windows Installer (.msi) 64-bit」を選びます。
4. ダウンロードした `node-v22.x.x-x64.msi` をダブルクリック。

5. インストーラは **基本すべて既定のまま「Next」でOK**。「Automatically install the necessary tools...」というチェック項目が出たら、チェックは外したままで問題ありません（Visual Studio Build Tools は本アプリのインストールには必須ではありません）。
6. 「Install」→「Finish」。

> ⚠️ **すでに別バージョンの Node が入っている人へ:** `node --version` で `v20.` で始まる番号が出るなら、そのままでも動きます（GitHub Actions のビルドも v20 です）。`v18.` 以前が出る場合は、古くて手入れが終わっているので入れ直します。複数バージョンを切り替えたい場合は「nvm-windows」というツールもありますが、初心者はまず **古い方をアンインストールして v22 を入れる** のが簡単です。

5.2 インストール確認

VSCode を再起動し、ターミナルで2つ打ちます。

> ▼ **このコマンドがやること:** Node.js と npm の版をそれぞれ表示します。

コピー

```
# Node.jsのバージョンを表示
node --version
# npm（部品管理ツール）のバージョンを表示
npm --version
```

> ▶ **ここで初めて出てくるコード**
>
> - **`node`**（Node.js）: Node.js（パソコンの上で JavaScript を動かす道具）を動かすコマンド。`node --version` で版を表示し、`node ファイル名.js` で、そのファイルを動かします。
> - **`node --version`**（Node.js）: `node --version` は、入っている Node.js の版を表示します。このアプリでは `v22.…`（おすすめ）か `v20.…` なら大丈夫です。
> - **`npm`**（Node.js）: Node.js の部品を入れる・動かす道具のコマンド。`npm --version` で版を確かめ、`npm install`（部品を入れる）・`npm start`（開発用のサーバーを動かす）・`npm run build`（本番用に作る）のように使います。このアプリでは、`frontend/` フォルダで打ちます。
> - **`npm --version`**（Node.js）: `npm --version` は、Node.js の部品を入れる道具 npm の版を表示します。

▼ こう表示されれば成功:

コピー

```
v22.17.1
10.9.2
```

`node` の方が `v22.` で始まっていれば成功です。`npm` の数字は版により異なります。

なぜ？ npm のバージョンは気にしなくていいの？ npm は Node.js に同梱され、Node のバージョンに対応した版が自動で入るためです。`node` が v22 なら、付属の npm をそのまま使えばOKです。

---

6. 道具をそろえる⑤：PostgreSQL

PostgreSQL（ポストグレス／ポストグレスキューエル）とは？ 無料で高性能な **データベース管理システム（DBMS）**。本アプリの工数・人員・チームなどの **データはすべてここに保存** されます。`settings.py`（§11）でも `'ENGINE': 'django.db.backends.postgresql'` と指定されており、本アプリは PostgreSQL 専用です。

データベース（database）とは？ データを整理して保存し、すばやく出し入れできる「巨大で賢い表計算ソフト」のようなもの。Excel の超強力版と思ってください。本アプリでは「工数の表」「人員の表」などが、この中にテーブル（表）として保存されます。

DBMS（ディービーエムエス）とは？ Database Management System（データベース管理システム）の略。データベースを動かす本体プログラム。PostgreSQL はその一種です。

6.1 インストール手順

1. ブラウザで `https://www.postgresql.org/download/windows/` を開く。
2. 「Download the installer」をクリック（EDB社の配布ページへ飛びます）。
3. 新しめのバージョン（例: 16.x）の「Windows x86-64」の **Download** を押す。
4. ダウンロードした `postgresql-16.x-windows-x64.exe` をダブルクリック。
5. インストーラを進めます。重要な画面だけ説明します。- **Installation Directory** … 既定のままでOK（`C:\Program Files\PostgreSQL\16`）。- **Select Components** … 全部チェックのままでOK（「pgAdmin 4」も入れておくと、後でデータを目で見られて便利）。- **Data Directory** … 既定のままでOK。- **Password** … ⚠️**ここで設定するパスワードは必ずメモしてください。**これが、データベースの管理者（ユーザー名 `postgres`）のパスワードになります。本章の `.env`（§10）でも使います。例として本書では `postgres` と仮定して説明しますが、**あなたが決めたものを使ってください。**- **Port** … 既定の `5432` のままにします（`settings.py` §11 も 5432 を前提）。- **Locale** … 既定（Default locale）でOK。
6. 確認画面で「Next」→ インストール完了まで待つ。
7. 最後に「Stack Builder を起動するか」と聞かれたら、**チェックを外して**「Finish」（追加コンポーネントは不要です）。

pgAdmin（ピージーアドミン）とは？ PostgreSQL を **マウスで操作する管理画面アプリ**。テーブルの中身を目で見たり、SQL（データベースへの命令文）を試したりできます。コマンドが苦手なうちは、これでデータを覗けると安心です。

6.2 インストール確認

PostgreSQL のコマンド `psql`（ピーエスキューエル）は、初期状態では PATH に入っていないことがあります。確認方法を2通り示します。

方法A（おすすめ・確実）：pgAdmin で確認 1. スタートメニューで「pgAdmin 4」を検索して起動。2. 初回はマスターパスワードを聞かれるので、任意に設定。3. 左の「Servers」→「PostgreSQL 16」をクリックし、**インストール時に決めたパスワード** を入力。4. 接続できて、左に「Databases」などが表示されれば、PostgreSQL は正常に動いています。

方法B：コマンドで確認（PATHに psql がある場合）

コピー

```
# PostgreSQLのクライアントバージョンを表示
psql --version
```

▶ ここで初めて出てくるコード

- **`psql`**（psql）: PostgreSQL に、コマンドでつないで SQL を打つ道具。`psql -h 127.0.0.1 -U postgres -d データベース名` でつなぎます（`-h` は相手の住所、`-U` は利用者の名前、`-d` はデータベース）。`\q` で抜けます。
- **`psql --version`**（psql）: `psql --version` は、psql（PostgreSQL につなぐ道具）の版を表示します。

▼ こう表示されれば成功:

コピー

```
psql (PostgreSQL) 16.3
```

`psql ... が認識されません` と出る場合は、PATH 未登録なだけで PostgreSQL 自体は入っています。方法A（pgAdmin）で確認すれば十分です。§12で必要になったら、`psql` のフルパス（例: `"C:\Program Files\PostgreSQL\16\bin\psql.exe"`）を直接打つ方法も紹介します。

7. ソースコードを手元に持ってくる（git clone）

道具がそろいました。いよいよ本アプリのソースコード（プログラム本体）を、あなたのPCにダウンロードします。

> **clone（クローン）とは？**「複製する」という意味。Git の `git clone` は、GitHub などに置かれた **リポジトリ（コードの倉庫）まるごと一式を、あなたのPCにコピー** するコマンドです。
>
> **リポジトリ（repository）とは？** ソースコードと、その全変更履歴をまとめて保管する「倉庫」。本アプリのコードもリポジトリとして管理されています。

7.1 置き場所（フォルダ）を決める

まず、コードを置くフォルダを作ります。本書の例では、引き継ぎ元と同じ `C:\Users\あなたのユーザー名\Documents\git-projects` を使います。

> **▼ このコマンドがやること:** Documents の下に `git-projects` フォルダを作り（既にあってもエラーにしない）、その中に移動します。

コピー

```
# 置き場フォルダを作成（-Force は既存でもエラーにしない）
mkdir "$HOME\Documents\git-projects" -Force
# そのフォルダに移動
cd "$HOME\Documents\git-projects"
```

> **▶ ここで初めて出てくるコード**
>
> - **`mkdir`**（PowerShell）:「make directory（フォルダを作る）」の略。`mkdir "$HOME\Documents\git-projects"` のように、途中のフォルダもまとめて作れます（`$HOME` はホームフォルダ `C:\Users\あなたの名前`）。もう同じ名前のフォルダがあると、エラーで知らせます（`-Force` を付けると、知らせずに進みます）。
> - **`mkdir -Force`**（PowerShell）: `mkdir フォルダ -Force` は、フォルダを作ります。`-Force` を付けると、もう有っても赤いエラーを出さずに進みます。
> - **`"…$変数…"`**（PowerShell）: `"…"`（ダブルクォート）で囲んだ文字の中の `$名前` は、変数の中身に置きかわります。`"こんにちは、$name さん"` → `こんにちは、太郎 さん`。そのまま `$` を出したい時は、`'…'`（シングルクォート）で囲みます。

- `mkdir` … Make Directory。フォルダ（ディレクトリ）を作るコマンド。
- `$HOME` … PowerShell で「自分のユーザーフォルダ（`C:\Users\あなた`）」を表す変数。
- `-Force` …「すでに同名フォルダがあってもエラーにしない」オプション。
- `cd` … Change Directory。指定フォルダへ移動するコマンド。

用語: ディレクトリ（directory） … 「フォルダ」と同じ意味。コマンドの世界では「ディレクトリ」と呼ぶことが多いです。`cd` の `d` も Directory の d。

7.2 clone する

▼ このコマンドがやること: 本アプリのリポジトリを、今いるフォルダの中にダウンロードします。`<リポジトリのURL>` は、引き継ぎ元（前任者）から教わった GitHub のURLに置き換えてください。

コピー

```
# リポジトリ一式をダウンロード（URLは前任者に確認）
git clone <リポジトリのURL>
```

▶ ここで初めて出てくるコード

- **`git clone`** （Git）: GitHub などにあるリポジトリ（プログラムの置き場所）を、丸ごと自分のパソコンに写してきます。`git clone <URL>` で、URL の最後の名前のフォルダができます。
- **`>`** （PowerShell）: `コマンド > out.txt` は、画面に出るはずの文を、`out.txt` というファイルに書き込みます（前の中身は消えます）。後ろに書き足したい時は `>>` を使います。

例（URLはダミーです。本物に置き換えてください）：

コピー

```
git clone https://github.com/your-company/hozen-kosu-React.git
```

▼ こう表示されれば成功:

コピー

```
Cloning into 'hozen-kosu-React'...
remote: Enumerating objects: 1234, done.
remote: Counting objects: 100% (1234/1234), done.
...
Receiving objects: 100% (1234/1234), 5.67 MiB | 3.21 MiB/s, done.
Resolving deltas: 100% (456/456), done.
```

完了すると、今いるフォルダの中に `hozen-kosu-React` というフォルダができます。

⚠️ **clone 時にユーザー名・パスワードを聞かれたら？** プライベート（非公開）リポジトリの場合、GitHub のログインが必要です。これは次章「02_Git_GitHub.md」で扱う「認証」の話です。とりあえずこの章では、前任者に「clone できる状態のURL（またはアクセス権）」を用意してもらってください。

7.3 プロジェクトフォルダを VSCode で開く

ダウンロードしたフォルダを VSCode で開きます。

コピー

```
# ダウンロードしたフォルダに移動
cd hozen-kosu-React
# 今いるフォルダをVSCodeで開く（.は「今のフォルダ」の意味）
code .
```

> **▶ ここで初めて出てくるコード**
>
> - `code`（コマンド）: VS Code を動かすコマンド。`code .` で今のフォルダを開き、`code --version` で版を確かめます。

- `code .` … VSCode で「今いるフォルダ」を開くコマンド。`.`（ドット）は「カレントディレクトリ（今いる場所）」を意味します。
- もし `code` コマンドが効かない場合は、VSCode のメニュー「ファイル」→「フォルダーを開く」から `hozen-kosu-React` を手で選んでも同じです。

> **▼ こう表示されれば成功**: VSCode の左の「エクスプローラー」に、`frontend`・`hozen_another`・`kosu`・`requirements.txt`・`manage.py` などのフォルダ／ファイルがずらりと並びます。これが本アプリの全ソースです。
>
> **用語: ルート（root）ディレクトリ** … プロジェクトの一番上のフォルダのこと。本アプリでは `hozen-kosu-React`（中に `manage.py` がある階層）が「バックエンドのルート」です。`CLAUDE.md` の「from root directory」はこの場所を指します。一方、フロント側のコマンドは `frontend/` フォルダの中で打ちます。**今どのフォルダにいるかを常に意識しましょう**（ターミナル左端の表示で分かります）。

---

8. バックエンド準備①：仮想環境（venv）を作る

ここからバックエンド（Django）の準備です。まず「仮想環境」を作ります。

仮想環境（virtual environment / venv）とは？ プロジェクトごとに、**専用の独立した Python の作業部屋** を作るしくみ。インストールした部品（Django など）を、その部屋の中だけに閉じ込めます。

たとえ話：仮想環境は「プロジェクト専用の引き出し」。Aプロジェクトの引き出しには Django 4.2、Bプロジェクトの引き出しには Django 5.0……と、**別々のバージョンを混ぜずに保管** できます。仮想環境を使わずにPC全体（システム）に入れてしまうと、別プロジェクトと部品が衝突して、どちらも壊れることがあります。

なぜ？ なぜわざわざ仮想環境を作るの？ 本アプリは Django **4.2.7**（§9参照）という特定の版を要求します。もしあなたのPCに別プロジェクト用の Django 5.x が入っていると衝突します。仮想環境を使えば、このプロジェクトだけに Django 4.2.7 を入れられ、他に影響しません。**プロの現場では仮想環境を使うのが常識です。**

8.1 venv を作る

⚠️ 必ず **プロジェクトのルート**（`manage.py` がある `hozen-kosu-React` フォルダ）にいる状態で実行します。

▼ このコマンドがやること: `venv` という名前の仮想環境（作業部屋）を、今のフォルダの中に作ります。

コピー

```
# 「venv」という名の仮想環境フォルダを作成
python -m venv venv
```

▶ ここで初めて出てくるコード

- **`python -m`**（Python の道具）: `python -m venv venv` の `-m` は「Python の中の道具（モジュール）を動かす」という指定。ここでは `venv`（仮想環境を作る道具）を動かして、`venv` という名前のフォルダに仮想環境を作ります。

分解すると： - `python` … Python を起動。 - `-m venv` …「venv モジュール（仮想環境作成機能）を実行せよ」という指定。`-m` は「モジュールを実行」の意味。 - 最後の `venv` … **作る仮想環境のフォルダ名**。慣習で `venv` という名前にします（別名でも可）。

▼ こう表示されれば成功: 数秒〜十数秒、無言で待ったあと、何も表示されずにプロンプトが戻ってきます（＝エラーなし）。VSCode 左のエクスプローラーに `venv` という新しいフォルダができていれば成功です。

用語: モジュール（module） … Python の「機能の部品」。`venv` は Python に最初から付いている標準モジュールの1つです。

8.2 venv を有効化（activate）する

作っただけでは使えません。「この部屋に入る」操作（有効化）が必要です。

▼ **このコマンドがやること:** 作った仮想環境を有効化し、「以後このターミナルでは、この部屋の Python／pip を使う」状態にします。

コピー

```
# 仮想環境を有効化（PowerShell用の有効化スクリプトを実行）
.\venv\Scripts\Activate.ps1
```

▶ **ここで初めて出てくるコード**

- `.\venv\Scripts\Activate.ps1` （Python の道具）: 仮想環境（このアプリ専用の Python の部品の置き場）に入る PowerShell のスクリプト。入ると、行の頭に `(venv)` と出ます。「スクリプトの実行が無効」と出たら、`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` で許可します。
- `.\` （PowerShell）: `.\venv\Scripts\Activate.ps1` の `.\` は、「今いるフォルダの中の」という意味です。PowerShell は、安全のため、今いるフォルダのプログラムやスクリプトでも `.\` を付けないと動かしません。

▼ **こう表示されれば成功:** プロンプトの先頭に `(venv)` という表示が付きます。

コピー

```
(venv) PS C:\Users\...\hozen-kosu-React>
```

この `(venv)` が、「**今あなたは仮想環境の中にいますよ**」という目印です。以後 `pip install` や `python manage.py ...` は、必ずこの `(venv)` が付いた状態で実行します。

⚠️ **「スクリプトの実行が無効になっているため…」というエラーが出たら**

コピー

```
.\venv\Scripts\Activate.ps1 : このシステムではスクリプトの実行が無効になっているため、ファイル ... を読み込むことができません。
```

これは PowerShell の安全設定（実行ポリシー）が厳しすぎるためです。次のコマンドで、**現在のユーザーに限り** スクリプト実行を許可します（安全な範囲の設定です）。

コピー

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned    # 自分のユーザーだけ、署名付き/ローカルのスクリプト実行を許可
```

「実行ポリシーを変更しますか？」と聞かれたら `Y` を打って `Enter` 。そのあと、もう一度 §8.2 の有効化コマンドを実行してください。

8.3 venv を抜ける（deactivate）には

作業を終えて部屋から出たいときは：

コピー

```
# 仮想環境を抜ける（(venv)の表示が消える）
deactivate
```

▶ ここで初めて出てくるコード

- `deactivate`（Python の道具）: 仮想環境から出るコマンド。行の頭の `(venv)` が消えます。

今は **抜けないでください**。`(venv)` が付いたまま、次の §9 に進みます。

---

9. バックエンド準備②：requirements.txt 全解説とインストール

`(venv)` が付いた状態のまま、Django などの部品を一括インストールします。何を入れるのかを、本物の `requirements.txt` を1行ずつ見てから実行しましょう。

9.1 requirements.txt とは

requirements.txt（リクワイアメンツ・テキスト）とは？「このプロジェクトに必要な Python パッケージと、その正確なバージョン」を1行1パッケージで列挙したファイル。`pip install -r requirements.txt` で、ここに書かれた全部を **一括インストール** できます。「必要な部品の買い物リスト」です。

パッケージ（package）／ライブラリ（library）とは？ 誰かが作って公開している「便利な機能の部品集」。自分でゼロから書かずに、これを取り込んで使います。`Django` も `pandas` も、すべてパッケージです。

9.2 本アプリの requirements.txt を1行ずつ読む

▼ **これから見るファイルがやること**: 本アプリのバックエンドが必要とする、24個のパッケージとその版を列挙しています。下記は `requirements.txt` の **実物そのまま** です。各行の右に、それが何者かを書きました。

コピー

```
Django==4.2.7                          # ① Web本体フレームワーク。本アプリの裏方の心臓
django-bootstrap-datepicker-plus==5.0.5  # ② 日付選択カレンダー部品（Django管理画面/フォーム用）
django-bootstrap4==23.3                # ③ Bootstrap4(見た目の枠組み)をDjangoで使う部品
django-crispy-forms==2.1               # ④ Djangoのフォームを綺麗に表示する部品
django-environ==0.11.2                 # ⑤ .envファイルから設定を読み込む部品（§10-11で核心）
django-filter==23.5                    # ⑥ 一覧の絞り込み（フィルタ）機能を簡単に作る部品
django-widget-tweaks==1.5.0            # ⑦ テンプレート内でフォーム部品の見た目を微調整する部品
django-widgets-improved==1.5.0         # ⑧ ⑦の系統。フォーム部品の改良
djangorestframework==3.14.0            # ⑨ REST API（フロントとデータをやり取りする窓口）を作る部品。最重要
django-q==1.3.9                        # ⑩ 非同期タスク（裏で時間のかかる処理）の実行基盤。qclusterの正体
gunicorn==21.2.0                       # ⑪ 本番でDjangoを動かすサーバー（ローカルでは使わない）
openpyxl==3.1.2                        # ⑫ Excel(.xlsx)ファイルを読み書きする部品
psycopg2-binary==2.9.9                 # ⑬ PythonからPostgreSQLに接続するためのドライバ。DB接続の必須部品
python-dateutil==2.8.2                 # ⑭ 日付計算を便利にする部品
python-dotenv==1.0.0                   # ⑮ .envを読む別系統の部品
python-json-logger==2.0.7              # ⑯ ログをJSON形式で出力する部品
xlwings==0.30.13                       # ⑰ ExcelをPythonから操作する部品
whitenoise==6.6.0                      # ⑱ 静的ファイル(CSS/JS/画像)を配信する部品。settings.pyに登場
beautifulsoup4==4.12.2                 # ⑲ HTMLを解析する部品（スクレイピング等）
pandas==2.1.2                          # ⑳ 表データを高速処理する定番部品（集計・加工）
python-Levenshtein==0.26.1             # ㉑ 文字列の似ている度合いを計算する部品
setuptools==76.1.0                     # ㉒ パッケージのビルド/インストール補助（土台ツール）
django-cors-headers==4.7.0             # ㉓ CORS(別オリジンからのアクセス許可)を設定する部品。settings.pyに登場
jsonfield==3.2.0                       # ㉔ モデルにJSON型のフィールドを持たせる部品
```

特に重要な5つを補足します（後の章や `settings.py` で再登場します）。

> **用語: REST API（レスト・エーピーアイ）** … フロントエンド（React）とバックエンド（Django）が **データをやり取りするための窓口・約束事**。⑨ `djangorestframework`（略称 DRF）がこの窓口を作ります。たとえば React が「工数データをください」と `/api/...` に頼むと、DRF が応答します。本アプリの心臓部です（`CLAUDE.md` の「REST API using Django REST Framework」に対応）。
>
> **用語: CORS（コアーズ）** … Cross-Origin Resource Sharing。「**別の住所（オリジン）から来たアクセスを許可するか**」を制御するブラウザの安全機能。本アプリは React が `localhost:3000`、Django が `localhost:8000` と **別の窓口（ポート）** で動くため、ブラウザは「別オリジン同士の通信」とみなしてブロックしようとします。㉓ `django-cors-headers` がこれを許可する設定を可能にし、`settings.py`（§11）で実際に許可しています。
>
> **用語: 非同期タスク（asynchronous task）** … 時間のかかる処理（バックアップ等）を、ユーザーを待たせずに **裏でこっそり実行** する仕組み。⑩ `django-q` がこれを担い、`qcluster`（§15）という別プロセスで動かします。`CLAUDE.md` の「Django-Q async tasks for backup/restore」に対応します。
>
> **用語: ドライバ（driver）** … 異なるもの同士をつなぐ「変換アダプタ」。⑬ `psycopg2-binary` は、Python と PostgreSQL の間の翻訳役（DBドライバ）。これがないと Django は PostgreSQL に接続できません。
>
> **用語: `==` の意味** … `Django==4.2.7` の `==` は「**ちょうどこの版を入れる**」という指定。`>=`（以上）などもありますが、本アプリは全部 `==` で版を固定しています。これにより「誰の環境でも同じ版が入る＝動作が再現する」を保証しています。

9.3 一括インストールを実行する

`(venv)` が付いていること、`requirements.txt` のあるルートフォルダにいることを確認してから実行します。

> **▼ このコマンドがやること:** `requirements.txt` に書かれた24パッケージを、仮想環境の中にまとめてダウンロード＆インストールします。

コピー

```
# requirements.txtの全部を仮想環境に一括インストール
pip install -r requirements.txt
```

> **▶ ここで初めて出てくるコード**
>
> - **`pip install`**（Python の道具）: Python の部品を入れます。`pip install 部品の名前` で 1 つ、`pip install -r requirements.txt` でファイルに書いた全部を入れます。
> - **`pip install -r`**（Python の道具）: `-r requirements.txt` は「このファイルに並んだ部品を全部入れる」という指定です。`requirements.txt` には `Django==4.2.7` のように、部品の名前と版が 1 行に 1 つずつ書いてあります。

- `pip install` … パッケージをインストールするコマンド。
- `-r requirements.txt` … `-r` は「Read（読む）」の意。指定ファイルに書かれた全部を入れる、の意味。

> **▼ こう表示されれば成功:** たくさんのダウンロード行が流れたあと、最後に次のような行が出ます。

コピー

```
Successfully installed Django-4.2.7 djangorestframework-3.14.0 django-q-1.3.9 psycopg2-binary-2.9.9 pandas-2.1.2 ... (以下略)
```

`Successfully installed ...` と出れば成功です。数十秒〜数分かかります（pandas など大きい部品があるため）。

> ⚠️ **`pip` を新しくするよう促すメッセージが出ても無視してOK:**

コピー

```
[notice] A new release of pip is available: 24.0 -> 24.x
```

これは単なるお知らせで、エラーではありません。気になれば `python -m pip install --upgrade pip` で更新できますが、必須ではありません。

> ⚠️ **`psycopg2` 関連でエラーが出た場合:** 本アプリは `psycopg2-binary`（ビルド済み版）を指定しているため、通常はそのまま入ります。万一エラーが出たら §21 を参照。

9.4 インストール結果の確認

入った主要パッケージを確認します。

コピー

```
# 仮想環境にインストール済みの全パッケージを一覧表示
pip list
```

▶ ここで初めて出てくるコード

- `pip list` （Python の道具）: 今の Python（仮想環境の中なら、その中）に入っている部品と版を一覧にします。

▼ こう表示されれば成功: `Django 4.2.7` 、`djangorestframework 3.14.0` 、`django-q 1.3.9` などが一覧に並びます。`requirements.txt` の中身と見比べてみましょう。

---

10. バックエンド準備③：.env を作る（SECRET_KEY生成含む）

Django を動かすには、秘密情報（パスワードなど）をまとめた `.env` （ドットイーエヌブイ）ファイルが必要です。これを自分で作ります。

`.env` ファイルとは？「環境変数（かんきょうへんすう）」をまとめて書いておくテキストファイル。データベースのパスワードや秘密鍵など、**コードに直接書きたくない秘密情報** を、ここに分けて保管します。ファイル名は拡張子なしの `.env` （ドットで始まる）です。

なぜ？ なぜ秘密情報をコードに直接書かないの？ コードは Git で共有・公開されることがあります。もしパスワードをコードに書くと、**全世界に秘密が漏れます**。だから秘密は `.env` に分け、`.env` 自体は Git で共有しない（`.gitignore` で除外する）のが鉄則です。`CLAUDE.md` にも「Backend requires `.env` file with: SECRET_KEY, DEBUG, DB_NAME, DB_USER, DB_PASSWORD, DB_HOST」と明記されています。

環境変数（environment variable）とは？「プログラムの外側から与える設定値」。プログラムは起動時にこれを読み込んで動作を変えます。`.env` は、この環境変数をファイルにまとめたものです。

10.1 SECRET_KEY を生成する

SECRET_KEY（シークレット・キー）とは？ Django が、セッションやパスワードの暗号化・改ざん検知に使う **秘密の鍵文字列**。長くてランダムな文字列にする必要があり、**絶対に他人に見せてはいけません**。新しい環境用には、新しい鍵を自分で生成します。

`(venv)` が付いた状態で、次を打って鍵を1つ生成します。

> **▼ このコマンドがやること:** Django の鍵生成機能を呼び出し、ランダムな SECRET_KEY を1個作って画面に表示します。

コピー

```
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

> **▶ ここで初めて出てくるコード**
>
> - **`python -c`** （Python の道具）: `python -c "…"` は、後ろの文字を Python のプログラムとして、その場で動かします。ファイルを作るほどでない、短い確かめに使います。

分解すると： - `python -c "..."` … `-c` は「次の文字列を Python のプログラムとして直接実行せよ」という指定。 - `from django.core.management.utils import get_random_secret_key` … Django の「ランダム鍵生成」機能を取り込む。 - `print(get_random_secret_key())` … 鍵を1個作って表示する。

> **▼ こう表示されれば成功:** 次のような、長くてランダムな文字列が1行表示されます（あなたの結果は毎回違います）。
>
> コピー
>
> ```
> k3$9wq!2zx@7v...（中略）...8nf#p1
> ```
>
> この **1行を丸ごとコピー** しておいてください。次の §10.2 で `.env` に貼り付けます。

10.2 .env ファイルを作って中身を書く

VSCode で、**プロジェクトのルート**（`manage.py` がある階層）に新しいファイル `.env` を作ります。

1. VSCode 左のエクスプローラーで、ルートフォルダ（`HOZEN-KOSU-REACT` など）の何もない所を右クリック →「新しいファイル」。
2. ファイル名を `.env`（先頭がドット、拡張子なし）として `Enter`。
3. 開いたファイルに、次の内容を書きます。

> **▼ このファイルがやること:** Django が起動時に読み込む秘密情報・設定をまとめます。`settings.py`（§11）がここから各値を取り出します。`<...>` の部分は、あなたの環境の値に置き換えてください。

コピー

```
# Django の秘密鍵（§10.1 で作った長い文字列を、' と ' で囲んで貼る）
SECRET_KEY='<§10.1で生成した長い文字列をそのまま貼る>'
# 開発中は True（詳しいエラー画面が出る）
DEBUG=True
# 使うデータベース名（§12 で作る）
DB_NAME=hozen_kosu
# PostgreSQL のユーザー名（既定は postgres）
DB_USER=postgres
# そのユーザーのパスワード（§6.1 でメモした値）
DB_PASSWORD=<PostgreSQLインストール時に決めたパスワード>
# データベースの場所（自分の PC ＝ localhost）
DB_HOST=localhost
# ヘルプ画面（データの初期化）を開く時のパスワード（手元では自分で決めてよい）
HELP_PATH=<ヘルプ画面のパスワード>
```

各行の意味（`settings.py` との対応は §11 で詳説）：- `# …` の行 … 説明（コメント）です。Django は読み飛ばします。**説明は、必ず値とは別の行に書きます**（理由はすぐ下の ⚠️）。- `SECRET_KEY='...'` … §10.1 で生成した鍵。`=` **の後に空白を入れず**、鍵を `'`（シングルクォート）で囲んで貼ります。鍵には `#` が入ることがあり、囲まないと `#` から後ろが説明だとみなされて消えてしまうからです。もし鍵が `$` で始まっていたら、§10.1 のコマンドでもう一度作り直してください（`$` で始まる値は「別の設定の名前」だと読まれ、起動の時に `Set the … environment variable` というエラーになります）。- `DEBUG=True` … 開発中は `True`。エラー時に原因が画面に詳しく出ます（後述）。- `DB_NAME=hozen_kosu` … 接続するデータベースの名前。本書では `hozen_kosu` という名前で作ります（§12）。好きな名前でも可ですが、§12と合わせてください。- `DB_USER=postgres` … PostgreSQL の管理ユーザー。インストール直後の既定は `postgres` です。- `DB_PASSWORD=...` … §6.1 でメモした、`postgres` ユーザーのパスワード。パスワードに `#` が入っている時は、`'` で囲みます（例: `DB_PASSWORD='ab#cd'`）。- `DB_HOST=localhost` … データベースが動いている場所。自分のPC内なので `localhost`。- `HELP_PATH=...` … ヘルプ画面（`/help`）でデータを初期化する時に聞かれるパスワード。無くても起動はしますが、ヘルプ画面でパスワードを送った時にエラーになります。手元では好きな文字でかまいません（本番の値は、引き継ぎの時に前任者から安全な方法で受け取ります。「000 引き継ぎ書」の §3）。

> ⚠️ **説明（`# …`）を値と同じ行に書かない。** `DB_NAME=hozen_kosu # 使うデータベース名` のように書くと、このアプリが `.env` を読む部品（django-environ 0.11）は `#` から後ろを消しますが、**その前の空白は値に残します**。値が「`hozen_kosu` ＋空白」になり、「そんな名前のデータベースは無い」というエラーで Django が動きません（実際に試して確かめました）。また、`#` を含む値は `'...'` で囲まないと `#` から後ろが消えます（`"..."` で囲んでも、うまく読めません）。

4. **`Ctrl + S` で保存** します。

> ⚠️ **`.env` は絶対に Git にコミット（公開）しないでください。**本アプリには通常 `.gitignore` に `.env` が登録されており、自動で除外されます。万一されていない場合は、前任者に確認してください。秘密の鍵やパスワードを含むためです。
>
> ⚠️ **`DEBUG` の値について:** `settings.py`（§11）では `DEBUG=env.bool('DEBUG')` と読み込まれます。`env.bool` は文字列 `True` / `False` を真偽値に変換するので、`.env` には **`True` または `False`** と書きます（`1` / `0` でも可）。**本番環境では必ず `False`** にしますが、ローカル開発では `True` にして詳しいエラーを見られるようにします。

---

11. settings.py を1行ずつ読む（.envとの対応）

ここで、Django の中枢設定ファイル `hozen_another/settings.py` を読み、**さっき作った `.env` の値が、どこでどう使われるか** を確認します。これが分かると「なぜ `.env` にあの項目が必要だったのか」が腑に落ちます。

> **settings.py（セッティングス・ピーワイ）とは？** Django の全設定をまとめた中枢ファイル。データベース接続・インストール済み機能・セキュリティ・言語などを決めます。`hozen_another/` フォルダの中にあります（`hozen_another` は本アプリの「プロジェクト設定」フォルダ名）。

11.1 冒頭：.env を読み込む部分（1〜15行目）

> ▼ **このコードがやること:** 必要な道具を取り込み、`.env` ファイルを読み込んで、`SECRET_KEY` と `DEBUG` をそこから取り出します。**ここが `.env` と settings.py の接続点** です。

コピー

```
# ファイルパス操作の道具を取り込む
from pathlib import Path
# OS(ファイル/環境変数)操作の道具を取り込む
import os
# .envを読むための道具（requirements.txtの⑤ django-environ）
import environ
# ログ出力の道具（後半で使用）
from logging.handlers import RotatingFileHandler

# プロジェクトのルートフォルダの絶対パスを計算
BASE_DIR = Path(__file__).resolve().parent.parent

# 環境変数を読み取る「読み取り器」を用意
env = environ.Env()
# ルートにある .env ファイルを読み込む
env.read_env(os.path.join(BASE_DIR, '.env'))

# .envの SECRET_KEY を取り出してセット（§10.1の鍵）
SECRET_KEY=env('SECRET_KEY')

# .envの DEBUG を真偽値(True/False)として取り出す
DEBUG=env.bool('DEBUG')
```

1行ずつ： - `from pathlib import Path` … ファイルやフォルダの場所（パス）を扱う標準道具を取り込みます。- `import os` … OS（基本ソフト）の機能（ファイル結合・環境変数など）を取り込みます。- `import environ` … `.env` を読むための部品（`requirements.txt` の⑤ `django-environ`）を取り込みます。**これが `.env` を読む主役です。** - `BASE_DIR = Path(__file__).resolve().parent.parent` … `__file__`（このファイル自身の場所）から **2つ上のフォルダ** ＝プロジェクトのルートを求めます。`.parent` が「1つ上」を意味し、2回書いて2つ上に行っています。- `env = environ.Env()` … 環境変数の「読み取り器」を1つ用意します。- `env.read_env(os.path.join(BASE_DIR, '.env'))` … ルート（`BASE_DIR`）にある `.env` を実際に読み込みます。`os.path.join` は「BASE_DIR と '.env' をつないで完全なパスを作る」道具。**これで §10 で作った `.env` の内容が Django に取り込まれます。** - `SECRET_KEY=env('SECRET_KEY')` … `.env` の `SECRET_KEY=...` の値を取り出してセット。**§10.2 で書いた鍵がここに入ります。** - `DEBUG=env.bool('DEBUG')` … `.env` の `DEBUG=True` を、文字列ではなく **真偽値**（True/False） として取り出します。だから §10 で `True` / `False` と書く必要がありました。

> **用語: 絶対パス（absolute path）** … `C:\Users\...\hozen-kosu-React` のように、ドライブから始まる「完全な住所」。対して「今いる場所からの相対的な住所」を相対パスといいます。`BASE_DIR` は絶対パスです。

11.2 許可するホストとアプリ一覧（19〜38行目）

> ▼ **このコードがやること:** どのドメインからアクセスを受け付けるか（ALLOWED_HOSTS）と、Django に組み込む機能（INSTALLED_APPS）を列挙します。

コピー

```
ALLOWED_HOSTS = [
    # 本番のドメイン
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    # 自分のPC（開発用）
    'localhost',
    # 自分のPCのIPアドレス表記
    '127.0.0.1'
    ]
```

- `ALLOWED_HOSTS` … 「このサーバーは、どの住所（ホスト名）宛のアクセスを受け付けるか」のリスト。`localhost` と `127.0.0.1` があるので、**あなたのPCでの開発アクセスは許可されています**（だから何も足さなくてOK）。
- `127.0.0.1` … `localhost` と同じ「自分自身」を表す数字のIPアドレス。

コピー

```
INSTALLED_APPS = [
    # Django標準の管理画面
    'django.contrib.admin',
    # 認証（ログイン）機能
    'django.contrib.auth',
    # モデル種別を扱う内部機能
    'django.contrib.contenttypes',
    # セッション（ログイン状態の保持）
    'django.contrib.sessions',
    # メッセージ表示機能
    'django.contrib.messages',
    # 静的ファイル(CSS/JS/画像)管理
    'django.contrib.staticfiles',
    # Bootstrap4部品（requirements③）
    'bootstrap4',
    # 日付選択部品（requirements②）
    'bootstrap_datepicker_plus',
    # 本アプリ本体！工数管理の自作アプリ
    'kosu',
    # 非同期タスク基盤（requirements⑩）
    'django_q',
    # REST API（requirements⑨ DRF）
    'rest_framework',
    # CORS許可（requirements㉓）
    'corsheaders',
]
```

- `INSTALLED_APPS` … Django に「使う機能（アプリ）」を登録するリスト。
- `'kosu'` … **本アプリの本体。**`CLAUDE.md` の「Django app: kosu - main application」に対応する自作部分です。
- `'rest_framework'`・`'django_q'`・`'corsheaders'` … §9で解説した重要パッケージが、ここで有効化されています。`requirements.txt`（入れる）と `INSTALLED_APPS`（使う）はセットで効きます。

11.3 CORS設定（53〜59行目）

> ▼ **このコードがやること:** React（localhost:3000）から Django（localhost:8000）への「別オリジン通信」を許可します。§9.2で説明したCORSの実設定です。

コピー

```
# すべてのオリジンからのアクセスを許可（開発向け）
CORS_ORIGIN_ALLOW_ALL = True
# 認証情報(Cookie)付きの通信を許可（ログイン状態を保つため）
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOW_ORIGINS = [
    # Reactの開発サーバー（フロント）
    'http://localhost:3000',
    # Django自身
    'http://localhost:8000',
    # Django自身（IP表記）
    'http://127.0.0.1:8000',
]
```

> ここで `http://localhost:3000`（React）が許可されているおかげで、**npm start した画面から Django にデータを取りに行けます**。この設定が無いと、ブラウザが通信をブロックし、画面にデータが出ません。

11.4 REST API のページング設定（61〜70行目）

コピー

```
REST_FRAMEWORK = {
    # ページ番号方式の分割
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    # 1ページあたり20件ずつ返す
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        # 受け取るデータはJSON形式
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        # 返すデータもJSON形式
        'rest_framework.renderers.JSONRenderer',
    ),
}
```

- `'PAGE_SIZE': 20` … 一覧データを **20件ずつ** に分けて返す設定。大量データを一度に返さず、ページ送りにします（フロントのページネーション `Components/Pagination` に対応）。
- `JSONParser` / `JSONRenderer` … フロントとのやり取りは **JSON**（後述）形式で統一、という指定。

> **用語: JSON（ジェイソン）** … JavaScript Object Notation。`{ "name": "山田", "age": 30 }` のような、人にもプログラムにも読みやすいデータの書き方。フロント（React）とバック（Django）は、このJSON形式でデータを交換します。

11.5 データベース設定（113〜122行目）← .envと直結

> ▼ **このコードがやること:** Django がどのデータベースに、どのユーザー・パスワードで接続するかを決めます。**ここで §10 の `DB_NAME` など `.env` の値が使われます。**この章の山場です。

コピー

```
DATABASES = {
    # 既定のデータベース接続
    'default': {
        # PostgreSQLを使う指定（§6で入れたもの）
        'ENGINE': 'django.db.backends.postgresql',
        # データベース名を環境変数DB_NAMEから取得
        'NAME': os.getenv('DB_NAME', ''),
        # ユーザー名を環境変数DB_USERから取得
        'USER': os.getenv('DB_USER', ''),
        # パスワードを環境変数DB_PASSWORDから取得
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        # 接続先ホストを環境変数DB_HOSTから取得
        'HOST': os.getenv('DB_HOST', ''),
        # ポート番号は5432固定（PostgreSQLの既定）
        'PORT': '5432',
    }
}
```

1行ずつ、`.env` との対応： - `'ENGINE': 'django.db.backends.postgresql'` ...「PostgreSQL を使う」固定指定。§6 で入れた PostgreSQL と、§9 の `psycopg2-binary`（ドライバ）があって初めて接続できます。 - `'NAME': os.getenv('DB_NAME', '')` ... `os.getenv('DB_NAME', '')` は「環境変数 `DB_NAME` を読む。無ければ空文字 `''`」の意味。**§10.2 で書いた `DB_NAME=hozen_kosu` がここに入ります。** - `'USER': os.getenv('DB_USER', '')` ... 同様に **`DB_USER=postgres`** が入る。 - `'PASSWORD': os.getenv('DB_PASSWORD', '')` ... **`DB_PASSWORD=...`**（あなたのパスワード）が入る。 - `'HOST': os.getenv('DB_HOST', '')` ... **`DB_HOST=localhost`** が入る。 - `'PORT': '5432'` ... §6.1 で既定の `5432` にしたポート。一致している必要があります。

> ⚠️ **超重要な気づき:** 上の `'NAME'` などは `os.getenv(...)` を使っています。`os.getenv` は **OSレベルの環境変数** を読みます。`.env` ファイルの内容は §11.1 の `env.read_env(...)` によって読み込まれ環境変数として展開されるため、`os.getenv('DB_NAME')` で `.env` の値を取得できます。**つまり `.env` に `DB_NAME` 等を正しく書いていないと、ここが空になり接続に失敗します。**接続エラー時はまず `.env` の綴りを疑ってください。

11.6 非同期タスク（django-q）の設定（124〜131行目）

> ▼ **このコードがやること:** §15で起動する `qcluster`（非同期ワーカー）の動作を決めます。

コピー

```
Q_CLUSTER = {
    # タスク情報を上のdefaultデータベースに保存する
    'orm': 'default',
    # 同時に働く作業者(ワーカー)を4人にする
    'workers': 4,
    # 2000タスクごとにワーカーを作り直す（メモリ掃除）
    'recycle': 2000,
    # 失敗時の再試行までの秒数
    'retry': 2000,
    # 1タスクの制限時間（1800秒=30分）
    'timeout': 1800,
}
```

- `'orm': 'default'` … 非同期タスクの記録を、上の `DATABASES['default']`（PostgreSQL）に保存。
- `'workers': 4` … 4つの処理を同時にこなせる、の意味。
- `'timeout': 1800` … 30分以内に終わらないタスクは打ち切り。バックアップ等の重い処理に対応した余裕ある設定です。

11.7 言語・タイムゾーン・静的ファイル（148〜172行目）

コピー

```
# 表示言語を日本語に
LANGUAGE_CODE = 'ja'
# 時刻を日本時間に
TIME_ZONE = 'Asia/Tokyo'
# 国際化(多言語)機能を有効に
USE_I18N = True
# 内部の時刻をタイムゾーン付きで扱う
USE_TZ = True

# 静的ファイル(CSS/JS/画像)のURLの先頭
STATIC_URL = '/static/'
# 集約された静的ファイルの保存先
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

- `LANGUAGE_CODE = 'ja'` / `TIME_ZONE = 'Asia/Tokyo'` … 日本語・日本時間の設定。管理画面が日本語になります。
- `STATIC_URL` / `STATIC_ROOT` … CSS・画像などの「静的ファイル」をどこに置き、どのURLで配るかの設定。本番では `whitenoise`（§9⑱）が配信します。

11.8 CSRFと静的ファイル保存方式（162〜168行目）

コピー

```
CSRF_TRUSTED_ORIGINS = [
  # 本番
  'https://hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
  # ローカルDjango
  'http://localhost:8000',
  # ローカルDjango（IP）
  'http://127.0.0.1:8000',
]
# 静的ファイルを圧縮配信
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

> **用語: CSRF（シーサーフ）** … Cross-Site Request Forgery（クロスサイト・リクエスト・フォージェリ）。「なりすまし送信」を防ぐ Django の安全機能。`CSRF_TRUSTED_ORIGINS` は「ここからの送信は信頼してよい」という許可リスト。ローカル開発の `localhost:8000` が含まれています。

これで `settings.py` の要点を読み終えました。**`.env`（§10）に書いた値が、`settings.py` のどこに入るのか** が見えたはずです。`.env` の項目を1つでも間違えると、対応する設定が空になって動かない——この対応関係を覚えておくと、トラブル時に必ず役立ちます。

---

12. バックエンド準備④：データベースを作って migrate

設定ができたので、いよいよデータベース本体を準備します。手順は2つ。**(A) 空のデータベースを作る → (B) Django に表（テーブル）を自動生成させる（migrate）**。

12.1 (A) 空のデータベースを作る

`.env` の `DB_NAME=hozen_kosu` に合わせて、`hozen_kosu` という名前の空データベースを PostgreSQL の中に作ります。pgAdmin（マウス）かコマンドの2通り。

方法A：pgAdmin で作る（初心者おすすめ） 1. pgAdmin 4 を起動し、`PostgreSQL 16` に接続。2. 左の「Databases」を **右クリック** →「**Create**」→「**Database...**」。3.「Database」欄に **`hozen_kosu`** と入力（`.env` の `DB_NAME` と完全一致させる）。4.「Save」。左に `hozen_kosu` が追加されれば成功。

方法B：コマンド（psql）で作る

コピー

```
# psqlにPATHが通っている場合
# postgresユーザーで hozen_kosu DBを作成
psql -U postgres -c "CREATE DATABASE hozen_kosu;"
```

▶ ここで初めて出てくるコード

- **`psql -U`**（psql）：`-U postgres` は、どの利用者（ロール）としてつなぐかです。パスワードを聞かれたら、その利用者のパスワードを入れます。
- **`psql -c`**（psql）：`-c "SELECT …;"` は、つないで、その SQL を 1 回だけ動かして、すぐに終わる指定です。確かめを 1 行で済ませる時に使います。
- **`CREATE`**（SQL）：`CREATE ROLE`（ユーザーを作る）・`CREATE DATABASE`（データベースを作る）・`CREATE TABLE`（表を作る）のように、何かを新しく作る SQL の命令です。
- **`DATABASE`**（SQL）：`CREATE DATABASE 名前` で、データベースを作ります。

- パスワードを聞かれたら、§6.1 で決めた `postgres` のパスワードを入力。
- `psql` が認識されない場合はフルパスで：

コピー

```
& "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -c "CREATE DATABASE hozen_kosu;"
```

▶ ここで初めて出てくるコード

- **`&`**（PowerShell）：`& "C:\Program Files\PostgreSQL\16\bin\psql.exe" …` の先頭の `&` は、「この文字が指すプログラムを動かす」という印です。場所に空白が入っていて `"` で囲んだ時に必要です（`&` が無いと、ただの文字として表示されるだけです）。

▼ こう表示されれば成功:

コピー

```
CREATE DATABASE
```

⚠️ **`database "hozen_kosu" already exists` と出たら**、すでに作成済みです。問題ありません。次へ進んでください。

12.2 (B) migrate でテーブルを作る

> **マイグレーション（migration）とは？** Django の「**モデル（データの設計図）から、データベースの表を自動で作る／更新する**」しくみ。`models.py`（`CLAUDE.md` の「Key Models」参照）に書かれた設計図どおりに、`member`・`Business_Time_graph` などの表を空のデータベースに作ってくれます。

`(venv)` が付いた状態で、ルート（`manage.py` がある場所）で実行します。

> **▼ このコマンドがやること:** モデルの設計図に従い、データベースに必要な全テーブルを作成します。

コピー

```
# 設計図どおりにデータベースの表を作成/更新する
python manage.py migrate
```

- `python manage.py` … Django の操作コマンドを呼び出す入口。`manage.py` は Django プロジェクトの「操作受付係」です。
- `migrate` …「マイグレーションを実行（＝表を作る・更新する）」というサブコマンド。

> **▼ こう表示されれば成功:** 次のような行が次々と表示されます。

コピー

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, django_q, kosu, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying kosu.0001_initial... OK
  ...
  Applying sessions.0001_initial... OK
```

各行の末尾が **`OK`** になっていれば成功です。これでデータベースの中に、本アプリのすべての表ができました。

> ⚠️ **よくあるエラーと対処（詳しくは §21）:** - `django.db.utils.OperationalError: connection ... failed` → `.env` の DB情報が違う、または PostgreSQL が起動していない／DBが未作成。 - `password authentication failed for user "postgres"` → `.env` の `DB_PASSWORD` が間違っている。 - `database "hozen_kosu" does not exist` → §12.1 のデータベース作成を忘れている。

---

13. バックエンド準備⑤：管理者を作る（createsuperuser）

データベースができたので、**管理者ユーザー（スーパーユーザー）** を1人作ります。これは Django の管理画面（`/admin`）にログインするためのアカウントです。

> **スーパーユーザー（superuser）とは？** すべての操作権限を持つ「最高管理者」アカウント。Django の管理画面で、データを直接見たり編集したりできます。開発時の確認に重宝します。
>
> **▼ このコマンドがやること:** 対話形式で質問に答えながら、管理者アカウントを1つ作成します。

コピー

```
# 管理者(スーパーユーザー)を作成する
python manage.py createsuperuser
```

実行すると、順に質問されます。

> **▼ こう表示されます（対話例）:**
>
> コピー
>
> ```
> ユーザー名 (leave blank to use 'yamada'): admin        ← 好きな管理者名を入力（例: admin）
> メールアドレス: you@example.com                          ← メール（空でもEnterで進める）
> Password: ********                                     ← パスワード入力（画面には何も出ない）
> Password (again): ********                             ← もう一度同じパスワード
> Superuser created successfully.                        ← これが出れば成功
> ```

入力のコツ： - **ユーザー名** … `admin` など分かりやすいものを。 - **パスワード** … 入力中、**画面には何も表示されません**（セキュリティのため）。打ち間違えに注意。 - 簡単すぎるパスワードだと「This password is too common.（一般的すぎる）」と警告が出ます。`Bypass password validation and create user anyway? [y/N]:` に `y` で強行もできますが、開発でも少し複雑なものにしましょう。

> **▼ こう表示されれば成功:**
>
> コピー
>
> ```
> Superuser created successfully.
> ```
>
> **後で使います:** このユーザー名とパスワードは、§14 で管理画面 `http://localhost:8000/admin/` にログインするのに使います。忘れないようメモを。

14. バックエンドを起動する（runserver）

ついにバックエンドを起動します！

> ▼ **このコマンドがやること:** Django の開発用サーバーを起動し、`http://localhost:8000` で待ち受け状態にします。

コピー

```
# Django開発サーバーを起動（localhost:8000で待機）
python manage.py runserver
```

> ▼ **こう表示されれば成功:**
>
> コピー
>
> ```
> Watching for file changes with StatReloader
> Performing system checks...
>
> System check identified no issues (0 silenced).
> June 02, 2026 - 10:30:00
> Django version 4.2.7, using settings 'hozen_another.settings'
> Starting development server at http://127.0.0.1:8000/
> Quit the server with CTRL-BREAK.
> ```
>
> 最後の `Starting development server at http://127.0.0.1:8000/` が出れば、**バックエンドは起動成功** です。
>
> ⚠️ **このターミナルは閉じないでください。** `runserver` は「起動しっぱなし」で動き続けます。サーバーが動いている間、このターミナルは入力を受け付けません（プロンプトが戻りません）。これが正常です。**別の作業は、新しいターミナルを開いて行います**（VSCodeでターミナル右上の「＋」または分割アイコン）。

14.1 動作確認：管理画面にログイン

ブラウザで `http://localhost:8000/admin/` を開きます。

> ▼ **こう表示されれば成功:** Django 管理サイトのログイン画面が出ます。§13 で作ったユーザー名・パスワードでログインすると、管理画面のトップ（`member`・`Business_Time_graph` などのモデル一覧）が表示されます。**ここまで来たら、バックエンドは完璧に動いています。**
>
> **用語: 開発サーバー**（development server） … `runserver` が起動する、**開発専用の簡易サーバー**。手軽ですが本番には使いません（本番は §9⑪ の `gunicorn` 等を使う）。ローカルで試すには十分です。

14.2 サーバーを止めるには

そのターミナルにフォーカスを当て、`Ctrl + C` を押します。プロンプトが戻れば停止です。再度起動したいときは、また `python manage.py runserver` を打ちます。

> ⚠️ **`Error: That port is already in use.` が出たら:** すでに別の `runserver` が 8000番で動いています。古い方を止める（`Ctrl + C`）か、`python manage.py runserver 8001` のように別ポートで起動します（ただしフロントの設定とずれるので、基本は古い方を止めるのが正解）。

---

15. 非同期ワーカーを起動する（qcluster）

本アプリは、バックアップ・復元などの **時間のかかる処理を裏で実行** します（`CLAUDE.md` の「Django-Q async tasks for backup/restore」）。それを担うのが `qcluster` です。

> **ワーカー（worker）とは？**「裏で黙々と仕事をこなす作業員プログラム」。`qcluster` は §11.6 の `Q_CLUSTER` 設定（ワーカー4人）に従って起動し、依頼された非同期タスクを順に処理します。

新しいターミナルを開き（runserver のターミナルとは別！）、`(venv)` を有効化してから実行します。

コピー

```
# （新ターミナルなので）仮想環境を再度有効化
.\venv\Scripts\Activate.ps1
# 非同期タスクのワーカー(Django-Q)を起動
python manage.py qcluster
```

▼ こう表示されれば成功:

コピー

```
10:32:00 [Q] INFO Q Cluster ... starting.
10:32:00 [Q] INFO Process ... ready for work at ...
...
10:32:00 [Q] INFO Q Cluster ... running.
```

`Q Cluster ... running.` が出れば、非同期ワーカーが待機状態です。これも **起動しっぱなし** で、このターミナルは閉じません。

なぜ？ qcluster は必ず起動が必要？ 普段のログインや工数入力だけなら、`qcluster` がなくても動きます。ただし、**バックアップ・復元など非同期タスクを使う機能** を試すときは、これが起動していないとタスクが処理されずに溜まったままになります。本番同様の動作を確認するなら、`runserver` と `qcluster` の **両方を起動** しておくのが安全です。

⚠️ ここまでで、ターミナルを **2つ** 使っています（①runserver、②qcluster）。次のフロントエンドで **3つ目** を開きます。VSCodeのターミナルは右上のドロップダウンで切り替えられます。

---

16. フロントエンド準備①：package.json 全解説と npm install

ここからフロントエンド（React）の準備です。まず、必要な部品リストである `package.json` を読みます。

package.json（パッケージ・ジェイソン）とは？ Node.js プロジェクトの **設定ファイル兼・部品リスト**。`requirements.txt` の Node.js 版にあたります。使う部品（dependencies）と、実行できるコマンド（scripts）が書かれています。`frontend/` フォルダの中にあります。

16.1 本アプリの package.json を読む

▼ **このファイルがやること:** フロントエンドの名前・部品（ライブラリ）・実行コマンドを定義します。下記は `frontend/package.json` の実物です。主要部を解説します。

コピー

```
{
  "name": "frontend",          // このプロジェクトの名前
  "version": "0.1.0",          // バージョン
  "private": true,             // 誤って公開(npm publish)しないための印
  "dependencies": {            // ▼アプリ動作に必要な部品（本番でも使う）
    "@emotion/react": "^11.14.0",          // MUIが内部で使うスタイル付け部品
    "@emotion/styled": "^11.14.1",         // 同上（スタイル付け）
    "@fullcalendar/daygrid": "^6.1.19",    // カレンダー表示部品(月表示)
    "@fullcalendar/interaction": "^6.1.19",// カレンダー操作部品
    "@fullcalendar/react": "^6.1.19",      // カレンダーのReact版
    "@fullcalendar/timegrid": "^6.1.19",   // カレンダー(時間軸表示)
    "@mui/material": "^7.2.0",             // Material-UI：ボタン等のUI部品集（CLAUDE.md記載）
    "@mui/x-date-pickers": "^8.11.2",      // MUIの日付選択部品
    "@testing-library/dom": "^10.4.0",     // テスト用のDOM操作部品
    "@types/react": "^19.1.7",             // ReactのTypeScript型定義
    "@types/react-dom": "^19.1.6",         // React-DOMの型定義
    "axios": "^1.11.0",                    // サーバーと通信する部品（API呼び出しの主役）
    "chart.js": "^4.5.0",                  // グラフ描画ライブラリ
    "chartjs-plugin-datalabels": "^2.2.0", // グラフに数値ラベルを出す拡張
    "date-fns": "^4.1.0",                  // 日付処理ライブラリ
    "dayjs": "^1.11.13",                   // 軽量な日付処理ライブラリ
    "react": "^19.1.0",                    // React本体！画面を作る主役（CLAUDE.md記載）
    "react-calendar": "^6.0.0",            // カレンダー部品
    "react-chartjs-2": "^5.3.0",           // chart.jsのReactラッパー
    "react-dom": "^19.1.0",                // ReactをブラウザDOMに描画する部品
    "react-router-dom": "^7.11.0",         // 画面遷移(ルーティング)の部品（CLAUDE.md記載）
    "react-scripts": "5.0.1",              // Create React Appの土台ツール（start/build等を提供）
    "typescript": "^4.9.5",                // TypeScript本体（型付きJavaScript）
    "web-vitals": "^2.1.4"                 // パフォーマンス計測部品
  },
  "scripts": {                 // ▼実行できるコマンド（npm run xxx で呼ぶ）
    "start": "react-scripts start",   // 開発サーバー起動（§18で使う）
    "build": "react-scripts build",   // 本番ビルド（§19で使う）
    "test": "vitest",                 // テスト実行（監視モード）
    "test:ui": "vitest --ui",         // テストをUIで実行
    "test:run": "vitest run",         // テスト1回実行
    "eject": "react-scripts eject"    // 設定を取り出す（通常使わない）
  },
  "eslintConfig": { ... },     // コードの書き方チェック(ESLint)の設定
  "browserslist": { ... },     // 対応ブラウザの範囲指定
  "devDependencies": {         // ▼開発時だけ使う部品（テスト等。本番には含めない）
    "@testing-library/jest-dom": "^6.9.1",   // テストの便利機能
    "@testing-library/react": "^16.3.1",     // Reactコンポーネントのテスト
    "@types/jest": "^30.0.0",                // jestの型定義
    "@vitejs/plugin-react": "^5.1.2",        // Vite用React対応
    "@vitest/coverage-v8": "^4.1.2",         // テストカバレッジ計測
    "@vitest/ui": "^4.0.16",                 // VitestのUI
    "eslint-config-react-app": "^7.0.1",     // ESLint設定
    "jsdom": "^27.3.0",                      // テスト用の擬似ブラウザ環境
    "vitest": "^4.0.16",                     // テスト実行ツール（CLAUDE.md記載）
    "webpack-bundle-analyzer": "^4.10.2"     // バンドルサイズ分析ツール
  }
}
```

▶ ここで初めて出てくるコード

- `"name":`（設定ファイル（JSON））：`package.json` の `"name"` は、このプロジェクト（パッケージ）の名前です。
- `"version":`（設定ファイル（JSON））：`package.json` の `"version"` は、このプログラム自身の版の番号です。
- `"private":`（設定ファイル（JSON））：`"private": true` は、「このパッケージは npm に公開しない」という印です。間違って公開するのを防ぎます。
- `"dependencies":`（設定ファイル（JSON））：`package.json` の `"dependencies"` は、アプリが動くのに必要な部品の一覧です。`"axios": "^1.11.0"` のように、部品の名前と版を書きます。`^` は「1.11.0 以上で、2.0.0 より前ならよい」という意味です。`npm install` で、ここに書いた部品が入ります。
- `"typescript":`（設定ファイル（JSON））：`"typescript": "^4.9.5"` は、TypeScript（型を確かめる道具）の部品と、その版です。`^` は「4.9.5 以上で、5.0.0 より前ならよい」という意味です。
- `"scripts":`（設定ファイル（JSON））：`"scripts"` には、`npm run 名前` で動かすコマンドを書きます。このアプリでは `"start": "react-scripts start"`（`npm start` で開発用のサーバー）・`"build": "react-scripts build"`（`npm run build` で本番用に作る）・`"test": "vitest"` などです。
- `"start":`（設定ファイル（JSON））：`"scripts"` の中の `"start"` は、`npm start` で動くコマンドです（ここでは `react-scripts start` ＝開発用のサーバーを起動）。
- `"build":`（設定ファイル（JSON））：`"scripts"` の中の `"build"` は、`npm run build` で動くコマンドです（本番用に、画面のファイルを `build/` フォルダにまとめる）。
- `"test":`（設定ファイル（JSON））：`"scripts"` の中の `"test"` は、`npm test` で動くコマンドです（ここでは `vitest` ＝ファイルを見張って、変わるたびにテストを流す）。
- `"test:ui":`（設定ファイル（JSON））：`"scripts"` の中の `"test:ui"` は、`npm run test:ui` で動くコマンドです（テストの結果をブラウザーの画面で見る）。
- `"test:run":`（設定ファイル（JSON））：`"scripts"` の中の `"test:run"` は、`npm run test:run` で動くコマンドです（テストを 1 回だけ流して終わる）。
- `"eject":`（設定ファイル（JSON））：`"scripts"` の中の `"eject"` は、react-scripts が隠している設定を取り出すコマンドです。元に戻せないので、ふだんは使いません。
- `"eslintConfig":`（設定ファイル（JSON））：`package.json` の `"eslintConfig"` は、ESLint（コードの書き方を調べる道具）の設定です。
- `"browserslist":`（設定ファイル（JSON））：`package.json` の `"browserslist"` は、どの版のブラウザーまで動くように作るかの指定です。`"production"`（本番のビルド用）と `"development"`（開発用）に分けて書きます。
- `"devDependencies":`（設定ファイル（JSON））：`"devDependencies"` には、開発の時だけ使う部品と、その版を書きます。`npm install` で入ります。

押さえるべきポイント：

> **用語: dependencies と devDependencies の違い** - `dependencies` （依存）... **アプリが動くのに必要** な部品。本番でも使う（React、axios など）。- `devDependencies` （開発依存）... **開発中だけ** 使う部品。テストツールなど、本番には含めない（vitest など）。 `npm install` （§16.2）は、**両方とも** インストールします。
>
> **用語: `^` （キャレット）の意味** ... `"react": "^19.1.0"` の `^` は「**19.1.0 以上で、メジャー番号19の範囲内の最新**」という許容指定。`requirements.txt` の `==` （完全固定）より少し緩い指定です。ただし実際に入る版は次の `package-lock.json` で固定されます。
>
> **用語: scripts** ... `npm run <名前>` で実行できる、登録済みコマンドの一覧。`start`・`build`・`test:run` などをこの後使います。`start` だけは慣習で `npm start` （ `run` 省略可）と書けます。
>
> **用語: axios（アクシオス）** ... サーバーと通信（HTTPリクエスト）するための定番ライブラリ。React から Django の `/api/...` にデータを取りに行く、その通信を担います。`CLAUDE.md` の「Frontend makes requests to /api/ endpoints」の実行役です。
>
> **用語: Material-UI / MUI（エムユーアイ）** ... `@mui/material` 。きれいなボタン・入力欄・表などの **既製UI部品集**。自分でCSSを書かなくても、整った見た目の部品を使えます（ `CLAUDE.md` の「Material-UI 7」）。

16.2 npm install を実行する

> ⚠️ **フロントのコマンドは、必ず `frontend/` フォルダの中で実行します。**ルートで打つと「package.json が見つからない」と怒られます。

新しいターミナルを開き（3つ目）、`frontend` に移動してからインストールします。

> **▼ このコマンドがやること:** `package.json` に書かれた全部品を、`frontend/node_modules` フォルダにまとめてダウンロード＆インストールします。

コピー

```
# フロントエンドのフォルダに移動（ルートにいる前提）
cd frontend
# package.jsonの全部品をnode_modulesにインストール
npm install
```

> **▶ ここで初めて出てくるコード**
>
> - **`npm install`** （Node.js）: `package.json` に書いてある部品（TypeScript など）を、まとめて入れるコマンド。入った部品は `node_modules` フォルダに入ります（Git には入れません）。最初の 1 回だけ打ちます。`npm install 部品の名前` なら、その部品を足します。

▼ **こう表示されれば成功**: たくさんの行が流れたあと、最後に次のように出ます（警告 `npm warn ...` は出ても問題ないことが多いです）。

コピー

```
added 1456 packages, and audited 1457 packages in 1m

250 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

`added ... packages` と出て、`node_modules` フォルダができていれば成功です。**初回は数分かかります**（部品が多いため）。

用語: node_modules（ノード・モジュールズ）とは？ `npm install` でダウンロードした全部品が入る巨大フォルダ。**サイズが非常に大きく、Git では共有しません**（`.gitignore` 済み）。だから clone 直後には存在せず、各自が `npm install` で作ります。これが、`requirements.txt` → `pip install` と同じ「リストから手元に部品を再現する」流れの Node 版です。

用語: package-lock.json とは？ `npm install` 時に自動生成・更新される「実際に入れた部品の正確な版を記録したファイル」。これがあると、誰がインストールしても **まったく同じ版** が入り、動作が再現します。`requirements.txt` の `==` 固定と同じ役割を、Node では `package.json`（範囲指定）＋ `package-lock.json`（実際の固定）で実現しています。

⚠️ **警告 `npm warn deprecated ...` が大量に出ても、ほとんどは無視してOK** です。`vulnerabilities`（脆弱性）が出ても、`found 0 vulnerabilities` なら問題なし。数字が出ても、初心者のうちは `npm audit fix` を **安易に実行しない** でください（依存が壊れることがあります）。

---

17. フロントエンド準備②：frontend/.env と REACT_APP_API_BASE_URL

React に「**バックエンド（Django）がどこにいるか**」を教える設定をします。`CLAUDE.md` の「Frontend uses `REACT_APP_API_BASE_URL` for API endpoint configuration」に対応する、フロント側の最重要設定です。

17.1 .env.production を読んで仕組みを理解する

本アプリの `frontend/` には `.env.production`（本番用設定）があります。まずこの実物を読み、設定の書き方を学びます。

▼ **このファイルがやること**: React が通信する相手（API）のベースURLを、環境（ローカル／ステージング／本番）ごとに切り替えるための設定です。下記は、GitHub の `master` ブランチにある `frontend/.env.production` の **実物そのまま** です（本番の行だけが有効）。

コピー

```
# ステージング環境にデプロイするときは下記を有効にしてビルド
# REACT_APP_API_BASE_URL=https://hozen-kosu-another-c6e2gyeraydpdnhq.japaneast-01.azurewebsites.net

# 本番環境にデプロイするときは下記を有効にしてビルド
REACT_APP_API_BASE_URL=https://hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net/


# ローカル環境での動作時は下記を有効にしてビルド
# REACT_APP_API_BASE_URL=http://localhost:8000
```

1行ずつ： - `# ...` で始まる行 … **コメント（無効化された行）**。`#` を付けると、その行は「メモ書き」になり設定としては効きません。ステージング用とローカル用の URL は、いまは `#` で無効化されています。 - `REACT_APP_API_BASE_URL=https://hozen-kosu-react-…/` … **唯一有効な行**。本番のサーバーの住所です。`master` に push すると、GitHub Actions が本番用の画面を作り（`npm run build`）、この住所が画面のプログラムの中に書き込まれます（09 章 §14）。

> **REACT_APP_API_BASE_URL とは？** React のコードから「API（バックエンド）の住所」を読み取るための環境変数。React（Create React App）では、**`REACT_APP_` で始まる名前** の環境変数だけがフロントのコードに渡される、というルールがあります。だからこの名前になっています。コード中では `process.env.REACT_APP_API_BASE_URL` で読み出し、axios の通信先に使います（`04` / `05` 章で実コードを確認）。
>
> ⚠️ **この行を切り替えたら、コミットする前に必ず戻す。** 手元で `npm run build` して、Django（`http://localhost:8000`）から画面を開いて試す時は、本番の行に `#` を付け、ローカルの行の `#` を外して作ります。**そのままコミットして `master` に push すると、本番の画面が「あなたの PC（localhost:8000）」を呼びに行き、だれもログインできなくなります。** コミットの前に `git status` を見て、`frontend/.env.production` が「modified」と出ていたら、`git diff frontend/.env.production` で中身を確かめ、本番の行が有効な形に戻します（02 章 §16 の `git restore` でも戻せます）。
>
> **なぜ？ なぜローカルの行は `http://localhost:8000` なの？** §14 で起動した Django が `localhost:8000` で待っているからです。画面が「工数データをください」と頼む相手＝Django の住所が、まさに `http://localhost:8000`。**ここがずれると、画面は出てもデータが取れません**（通信先が間違うため）。ふだんの開発（`npm start`）では、このファイルではなく、次の §17.2 で作る `frontend/.env` が使われます。

17.2 ローカル用の frontend/.env を用意する

開発（`npm start`）では `.env`（または `.env.development`）が使われます。**ローカル開発用に、`frontend/.env` を作成（または確認）** します。

VSCode で `frontend/` フォルダの中に `.env` という新しいファイルを作り、次の1行を書きます。

> ▼ **このファイルがやること:** ローカル開発時、React が通信する Django の住所を `localhost:8000` に設定します。

コピー

```
# 開発の時の API の接続先（自分の PC の Django）
REACT_APP_API_BASE_URL=http://localhost:8000
```

保存（`Ctrl + S`）します。

> ⚠️ **ここでも、説明（`# …`）は別の行に書きます。** `REACT_APP_API_BASE_URL=http://localhost:8000 # 説明` のように同じ行に書くと、React の部品（react-scripts 5 の中の dotenv 10.0.0）は、`#` から後ろまで全部を値として読みます。接続先が「`http://localhost:8000 # 説明`」という壊れた住所になり、ログインのボタンを押しても何も返ってきません（実際に試して確かめました）。
>
> ⚠️ **`.env` を編集・新規作成したら、`npm start` を再起動する必要があります。** React の開発サーバーは **起動時に一度だけ** 環境変数を読み込むため、起動中に `.env` を変えても反映されません。すでに起動していたら、一度 `Ctrl + C` で止めて、もう一度 `npm start` してください（§18）。
>
> **用語: `.env` / `.env.development` / `.env.production` の使い分け（React/CRAの場合）** - `.env` … 常に読まれる共通設定。- `.env.development` … `npm start`（開発）のときに追加で読まれる。- `.env.production` … `npm run build`（本番ビルド §19）のときに読まれる。ローカルで `npm start` する分には、`frontend/.env` に `REACT_APP_API_BASE_URL=http://localhost:8000` があれば十分です。

---

18. フロントエンドを起動する（npm start）

すべての準備が整いました。フロントエンドを起動します。

> ⚠️ **前提：バックエンド（§14 runserver）が起動していること。** 画面だけ起動してもデータは取れません。`runserver`（ターミナル①）が動いている状態で進めてください。

`frontend/` フォルダにいることを確認して実行します。

> ▼ **このコマンドがやること:** React の開発サーバーを起動し、`http://localhost:3000` でアプリ画面を表示します。`package.json` の `"start": "react-scripts start"`（§16.1）が実行されます。

コピー

```
# React開発サーバーを起動（localhost:3000）
npm start
```

▶ ここで初めて出てくるコード

- **`npm start`**（Node.js）：`package.json` の `"scripts"` の `"start"` を動かします。このアプリでは、画面の開発用サーバーが起動し、ブラウザーで `http://localhost:3000` が開きます。止める時は `Ctrl` ＋ `C` です。

▼ こう表示されれば成功:

コピー

```
Compiled successfully!

You can now view frontend in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.x.x:3000

Note that the development build is not optimized.
To create a production build, use npm run build.
```

自動でブラウザが開き、`http://localhost:3000` に **本アプリのログイン画面** が表示されれば、**フロントエンドの起動成功** です！（自動で開かなければ、ブラウザで `http://localhost:3000` を手で開く。）

▼ 最終ゴール確認: ログイン画面で、業務工数システムのロゴと「従業員番号」の入力欄が見えれば、フロント・バック・DBが全部つながっています。**おめでとうございます。環境構築は完了です。**

18.1 動作確認のチェックリスト

確認項目	OKの状態
ターミナル① runserver	`Starting development server at http://127.0.0.1:8000/` 表示中
ターミナル② qcluster	`Q Cluster ... running.` 表示中
ターミナル③ npm start	`Compiled successfully!` 表示中
ブラウザ `localhost:3000`	ログイン画面が表示される
ブラウザ `localhost:8000/admin/`	管理画面にログインできる

⚠️ **画面は出るがデータが出ない／ログインできないとき:** ほぼ「フロントの API 接続先（§17）」か「CORS（§11.3）」か「バックエンド未起動（§14）」のいずれかです。ブラウザの **F12 → Console / Network** タブでエラーを確認します（`03` 章§14、`07` 章で詳説）。`401`（未認証）や `Network Error`、`CORS` の文字がヒントになります。

18.2 ホットリロード（自動再読み込み）

`npm start` 中にコードを保存すると、ブラウザが自動で更新されます。これを **ホットリロード（hot reload）** といいます。いちいちブラウザを手動更新しなくても、変更が即反映される開発者にやさしい仕組みです。

18.3 止めるには

`npm start` のターミナルで `Ctrl + C`。「バッチ ジョブを終了しますか (Y/N)?」と出たら `Y`。

---

19. 本番ビルド（npm run build）

最後に、**本番用の成果物を作る** コマンドを紹介します（普段の開発では使いませんが、デプロイ時に必須なので仕組みを知っておきます）。

> **ビルド（build）とは？** 開発用に書かれた React/TypeScript のコードを、**ブラウザが効率よく読める形に変換・圧縮** して、配布用ファイル一式を作ること。「材料（ソース）を、出荷できる製品（静的ファイル）に組み立てる」工程です。
>
> **▼ このコマンドがやること:** `frontend/build/` フォルダに、本番配信用の HTML・CSS・JavaScript（圧縮済み）を生成します。`package.json` の `"build": "react-scripts build"`（§16.1）が実行されます。

コピー

```
# 本番用の静的ファイル一式を frontend/build に生成
npm run build
```

> **▶ ここで初めて出てくるコード**
>
> - **`npm run`**（Node.js）：`package.json` の `"scripts"` に書いたコマンドを動かすコマンド。このアプリでは `npm run build`（本番用のファイルを `frontend/build/` に作る）・`npm run test:run`（テストを 1 回流す）などです（`start` と `test` だけは `run` を省けます）。止める時は `Ctrl` + `C` です。

▼ こう表示されれば成功:

コピー

```
Creating an optimized production build...
Compiled successfully.

File sizes after gzip:

  234.56 kB  build/static/js/main.xxxxxxx.js
   12.34 kB  build/static/css/main.xxxxxxx.css
  ...

The build folder is ready to be deployed.
```

`The build folder is ready to be deployed.` が出れば成功。`frontend/build/` に成果物ができます。

19.1 build と settings.py のつながり

ここで `settings.py`（§11）の次の部分を思い出してください。

コピー

```
STATICFILES_DIRS = [
    # ビルドした静的ファイルの場所
    os.path.join(BASE_DIR, 'frontend/build/static'),
]
# Reactのbuild内のindex.htmlをテンプレートとして指定
# ビルドのindex.htmlを使う
TEMPLATES[0]['DIRS'] = [os.path.join(BASE_DIR, 'frontend/build')]
```

> **これが重要な設計です。**本番では、`npm run build` で作った `frontend/build/` を **Django が配信** します。`STATICFILES_DIRS` で `build/static`（CSS/JS）を、`TEMPLATES[0]['DIRS']` で `build/index.html` を Django に教えています。つまり **本番は「Django 1つが、画面（Reactのビルド成果）もAPIも両方配信する」** 構成です。ローカル開発では別々（3000と8000）に動かしますが、本番では合体する——この違いを覚えておくと、`08` 章のデプロイで迷いません。
>
> ⚠️ **ローカル開発では `npm run build` は不要です。**普段は `npm start`（開発サーバー）で十分。`build` は「本番に出す」ときだけ実行します。
>
> **`REACT_APP_API_BASE_URL` の切り替え（§17.1の再確認）**：本番ビルド時は `.env.production` が読まれます。`master` にある `.env.production` は、はじめから本番用の URL が有効です（本番用の画面は、`master` に push した時に GitHub Actions が作ります）。手元でビルドを試すためにローカルの行へ切り替えた時は、**コミットする前に必ず本番の行に戻します。ふだんの開発（`npm start`）だけなら、このファイルは触らなくて OK** です。

---

20. 毎日の起動手順まとめ（チートシート）

環境構築は1回きりですが、**毎日の開発では「起動」だけを繰り返します。**最短手順をまとめます。VSCodeでプロジェクトを開いたら、ターミナルを3つ開いて：

ターミナル①（バックエンド）

コピー

```
# 仮想環境を有効化
.\venv\Scripts\Activate.ps1
# Django起動（localhost:8000）
python manage.py runserver
```

ターミナル②（非同期ワーカー）

コピー

```
# 仮想環境を有効化
.\venv\Scripts\Activate.ps1
# 非同期ワーカー起動
python manage.py qcluster
```

ターミナル③（フロントエンド）

コピー

```
# フロントのフォルダへ
cd frontend
# React起動（localhost:3000）
npm start
```

→ ブラウザで `http://localhost:3000` を開く。終わるときは各ターミナルで `Ctrl + C`。

> **初回だけ必要だった作業（毎日は不要）**：各ツールのインストール（§2-6）、`git clone`（§7）、`python -m venv venv`（§8.1）、`pip install -r requirements.txt`（§9）、`.env` 作成（§10・§17）、DB作成＋`migrate`（§12）、`createsuperuser`（§13）、`npm install`（§16）。これらは原則 **1回やれば再実行不要** です（`requirements.txt` や `package.json` が更新されたら、再度 `pip install -r requirements.txt` / `npm install` を実行）。

---

21. トラブルシューティング

症状から原因を引ける一覧です。多くは「PATH」「venv未有効化」「フォルダ違い」「.env の綴り」「バージョン違い」のどれかです。

21.1 ツールのインストール・コマンド系

症状	原因	対処
`'git'/'python'/'node'/'npm' は認識されません`	PATH未登録、またはインストール後にVSCode未再起動	インストール時に「Add to PATH」にチェックしたか確認。VSCodeを閉じて開き直す。直らなければ入れ直し
`python` と打つとMicrosoft Storeが開く	Windowsのアプリ実行エイリアスが有効	Windows設定→「アプリ」→「アプリ実行エイリアス」→「アプリ インストーラー python.exe / python3.exe」を **オフ**。VSCode再起動
`python --version` が3.13 など想定外	複数のPythonが入っている／別版を入れた	3.12系を入れ直す。`py -3.12 --version` で3.12が呼べるか確認し、venv作成も `py -3.12 -m venv venv` に
`node --version` が v18 以前	古いNodeが入っている	古い方をアンインストールし、v22 LTSを入れ直す（v20ならそのままで可）
`psql は認識されません`	PostgreSQLのbinがPATHにない（PostgreSQL自体は入っている）	pgAdminで確認すればOK。コマンドが要るときはフルパス `"C:\Program Files\PostgreSQL\16\bin\psql.exe"` で実行

21.2 仮想環境（venv）系

症状	原因	対処
`Activate.ps1 ... スクリプトの実行が無効`	PowerShellの実行ポリシーが厳格	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` を実行し `Y`。その後 再度 activate（§8.2）
プロンプトに `(venv)` が付かない	有効化に失敗、または別ターミナル	ルートで `.\venv\Scripts\Activate.ps1` を実行。新しいターミナルでは毎回有効化が必要
`pip install` したのに `ModuleNotFoundError`	`(venv)` 無しでインストール／実行している	`(venv)` を付けてから `pip install` と `python manage.py ...` を実行
`python -m venv venv` でエラー	Pythonが正しく入っていない	§4を再確認。`python --version` が3.12系か確認

21.3 バックエンド（Django）系

症状	原因	対処
`KeyError: 'SECRET_KEY'` / `ImproperlyConfigured`	`.env` が無い、または `SECRET_KEY` 未記入	ルートに `.env` があるか、`SECRET_KEY=...` を書いたか確認（§10）
`OperationalError: connection ... failed`	PostgreSQL未起動／DB未作成／`.env` のDB情報違い	サービスでPostgreSQL起動確認。`hozen_kosu` DBを作ったか（§12.1）。`.env` のDB_HOST/NAME確認
`password authentication failed for user "postgres"`	`.env` の `DB_PASSWORD` が間違い	§6.1で決めたパスワードを `.env` に正しく記入。pgAdminで同じパスワードで繋がるか確認
`database "hozen_kosu" does not exist`	DB未作成	§12.1 でデータベースを作成
`migrate` で `No module named 'psycopg2'` 等	DBドライバ未インストール	`(venv)` 有効化後 `pip install -r requirements.txt` を再実行
`runserver` で `That port is already in use`	8000番が使用中（別のrunserver）	古いrunserverを `Ctrl+C`。または `runserver 8001`（基本は古い方を止める）
`createsuperuser` でパスワードが弱いと拒否	パスワード検証	少し複雑なパスワードに。または警告に `y` で強行（開発のみ）
管理画面が英語	（実は日本語設定済み）キャッシュ等	`LANGUAGE_CODE='ja'` は設定済み。ブラウザを更新／別ブラウザで確認

21.4 フロントエンド（React）系

症状	原因	対処
`npm install` で `package.json not found`	`frontend/` 以外で実行している	`cd frontend` してから `npm install`（§16.2）
`npm install` が途中で失敗	ネットワーク／キャッシュ破損／Node版違い	Node が v20 か v22 か確認。`node_modules` と `package-lock.json` を削除し再 `npm install`。社内プロキシなら設定確認
`npm start` でブラウザは出るがデータが出ない	API接続先違い／バックエンド未起動／CORS	`frontend/.env` の `REACT_APP_API_BASE_URL=http://localhost:8000` 確認（§17）。runserver起動確認（§14）。F12 ConsoleでCORS/Network Error確認
`.env` を変えたのに反映されない	npm startが古い設定のまま	`Ctrl+C` で止めて `npm start` を再起動（§17.2）
`localhost:3000` でログインしても弾かれる	セッション/Cookie・CORS_ALLOW_CREDENTIALS	バックエンド側 §11.3 の設定は済み。ブラウザのCookieブロックやシークレットモードを疑う
`Port 3000 is already in use`	3000番が使用中（別のnpm start）	古い方を止める。`Y` で別ポート起動を提案されたら従ってもよい
大量の `npm warn deprecated`	依存の警告（多くは無害）	`found 0 vulnerabilities` なら無視。安易に `npm audit fix --force` しない

21.5 「今どこにいるか」が分からなくなったら

コピー

```
# 今いるフォルダ（カレントディレクトリ）の絶対パスを表示
pwd
# 今いるフォルダの中身一覧（manage.pyがあればルート、package.jsonがあればfrontend）
ls
# 1つ上のフォルダへ移動
cd ..
```

> ⚠️ **コマンドを打つ前に必ず「今いる場所」を確認する習慣を。**バックエンド系（`python manage.py ...`）は**ルート**（`manage.py` のある所）で、フロント系（`npm ...`）は **`frontend/`** で打ちます。場所違いはエラーの最頻出原因です。

---

22. 演習問題

手を動かすと定着します。**自分のローカル環境**（壊しても他に影響しない）で試してください。

1. **バージョン確認の練習：** `git --version` / `python --version` / `node --version` / `npm --version` を順に打ち、それぞれが「Git 2.x」「Python 3.12.x」「Node v22.x（v20.x でも可）」「npm 10.x」相当になっていることを確認せよ。1つでも想定外なら、対応する §（3/4/5）に戻る。
2. **仮想環境の出入り：** `(venv)` が付いていない状態で `pip list` を実行し、付いている状態で `pip list` を実行して、表示される内容が違うことを確認せよ。`deactivate` と `.\venv\Scripts\Activate.ps1` を切り替えて、プロンプトの `(venv)` 表示が出たり消えたりするのを観察せよ。
3. **SECRET_KEY をもう1つ生成：**§10.1 のコマンドをもう一度実行し、**毎回違うランダム文字列** が出ることを確認せよ（※ `.env` は書き換えないこと。観察のみ）。
4. **settings.py と .env の対応を追う：** `settings.py` の `'NAME': os.getenv('DB_NAME', '')`（§11.5）と、`.env` の `DB_NAME=hozen_kosu`（§10.2）が対応していることを、両ファイルを並べて目で確認せよ。`DB_NAME` を一時的に存在しない名前にして `python manage.py migrate` を実行し、`database "..." does not exist` エラーが出ることを確認したら、**必ず元に戻す**。
5. **3つのターミナルでフル起動：**§20 のチートシートに従い、runserver・qcluster・npm start の3つを同時に起動し、`localhost:3000` でログイン画面、`localhost:8000/admin/` で管理画面が両方開けることを確認せよ。
6. **API接続先を壊して直す：** `frontend/.env` の `REACT_APP_API_BASE_URL` を `http://localhost:9999`（存在しない）に変え、`npm start` を **再起動** して、ログインなどでデータが取れなくなる（エラーになる）ことをF12 Consoleで確認せよ。確認後、`http://localhost:8000` に戻して再起動し、直ることを確認せよ。
7. **本番ビルドを覗く：** `cd frontend` して `npm run build` を実行し、`frontend/build/` フォルダができることを確認せよ。`build/static/js/` の中に圧縮された `.js` ファイルが生成されているのを見て、§19.1 の「本番ではこれを Django が配信する」を実感せよ。

---

23. この章のまとめ

- 本アプリは **フロントエンド**（React＝Node.js製、localhost:3000）と **バックエンド**（Django＝Python製、localhost:8000）の **二刀流構成**。データは PostgreSQL に保存される。

- 必要な道具は5つ：**VSCode（机）**／**Git（取得・履歴）**／**Python 3.12（裏方の言語）**／**Node.js v22（画面側の言語）**／**PostgreSQL（倉庫）**。各インストール後は `--version` で確認し、**「Add to PATH」のチェック**を忘れない。
- コードは **`git clone`** で手元に取得。
- バックエンドは **`python -m venv venv`** → **`Activate.ps1`**（**`(venv)` 確認**）→ **`pip install -r requirements.txt`** の順。`requirements.txt` は Django・DRF・django-q・psycopg2・pandas など24パッケージの「買い物リスト」。
- **`.env`** に `SECRET_KEY`（生成）・`DEBUG`・`DB_NAME`・`DB_USER`・`DB_PASSWORD`・`DB_HOST` を書く。これを **`settings.py`** が読み込み、`DATABASES` 等で使う。**`.env` の綴りミスは接続失敗の最頻出原因。**
- DBは **空のDBを作成** → **`migrate`**（**表を自動生成**）→ **`createsuperuser`**（**管理者作成**）。
- 起動は **`runserver`**（8000） ＋ **`qcluster`**（**非同期**） ＋ フロントの **`npm install`** → **`npm start`**（3000）。フロントの接続先は **`frontend/.env`** の **`REACT_APP_API_BASE_URL=http://localhost:8000`**。
- 本番では **`npm run build`** の成果物（`frontend/build/`）を **Django が配信**（`settings.py` の `STATICFILES_DIRS` / `TEMPLATES['DIRS']`）。ローカル開発では build 不要。
- 困ったら **PATH**／**`(venv)`**／**今いるフォルダ**／**`.env` の綴り**／**バージョン** の5点を疑う。

次は、コードの変更履歴を安全に管理する道具、「02_Git_GitHub.md」へ進みます。`git clone` で取得したこのコードを、今度は **自分で変更・記録・共有** できるようになりましょう。